UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

Examen Parcial 1
Software de Incidencias en la UAGRM

Estudiante: Choque Calle Jorge

Registro: 219074240

Materia: Arquitectura de Software

Sigla: INF 511

Docente: Ing. Josue Obed Veizaga Gonzales

Semestre 1/2026

Santa Cruz De la Sierra – Bolivia

Contenido

CAPITULO 1: REQUISISTOS 4
1.1. Requisitos Funcionales 4
1.2. Requisitos No Funcionales 5
1.3. Identificar Actores 5
1.4. Identificar Casos de Uso 7
1.5. Priorizar Casos de Uso 8
1.6. Detallar Casos de Uso 9
CU – 01: Inicio de sesión 9
CU-02: Cierrar de sesión 11
CU-03: Registro de los reportadores de incidencia 13
CU-04:Gestion de usuarios 15
CU-05:gestionar infraestructura física 17
CU-06:registro de incidencia 21
CU-07: gestionar facultades 23
CU08:registro de prueba de evidencia 25
CU09:asignación de personal a incidencia 27
CU10:visualizar incidencia 30
CU11:finalizar orden de mantenimiento 32
CU12:consulta de historial de incidencia 35
1.7. Diagrama general de Casos de Uso 37
CAPITULO 2: ANALISIS 38
2.1. identificación de Paquetes 38
2.2. relacionar paquetes con Casos de Uso 39
CAPTULO 3: DISEÑO 42
3.1. Diseño de la Arquitectura 42
3.2. Diseño de la Base de Datos 43
3.2.1. Diseño conceptual 43
3.2.2. Diseño Logico 47
3.2.3. Diseño Fisico 50
3.3. Diseño de detalle procidemental 53
CU – 01: Inicio de sesión 53

CU-02: Cierre de sesión 56
CU-03: Registro de los reportadores de problemas 57
CU-04:Gestion de usuarios 60
CU-05:gestionar estructura física 62
CU-06:registro de notas de problemas 64
CU-07:gestionar facultades 66
CU08:registro de evidencia 68
CU09:asignación de problema 70
CU10:visualizar problemas 71
CU11:Finalizar mantenimiento 73
CU12:consulta de historial 77
Código 79

CAPITULO 1: REQUISISTOS

El software Permitra que la comunidad universitaria (estudiantes, docentes y administrativos) reporte anomalías en servicios críticos como climatización, redes eléctricas, fontanería y mobiliario, asegurando que cada reporte sea canalizado a las autoridades y técnicos correspondientes para su pronta resolución.

1.1. Requisitos Funcionales

RF - 01: Administración de Informantes

El sistema deberá permitir el registro y la validación de los datos de contacto de cualquier miembro de la comunidad que identifique una anomalía en el campus.

RF - 02: Ciclo de Vida de Tickets de Incidencia

La plataforma debe garantizar la gestión integral (creación, edición, consulta y cierre) de las solicitudes de reparación, manteniendo la integridad de la información en cada etapa.

RF - 03: Georreferenciación de Infraestructura

Se requiere un desglose jerárquico para la ubicación de las fallas, permitiendo especificar la Facultad, el Bloque/Módulo, el Nivel y el número de Aula u oficina afectada.

RF - 04: Control de Especialistas de Mantenimiento

El software permitirá la administración del personal técnico especializado, facilitando su alta, baja y actualización de datos para la asignación de tareas.

RF - 05: Repositorio de Evidencia Visual

Se debe integrar la capacidad de adjuntar archivos multimedia (fotografías o capturas) que sirvan como prueba documental del estado de la incidencia antes y después de la intervención.

RF - 06: Módulo de Supervisión de Autoridades (Decanos)

El sistema permitirá gestionar a los Decanos de cada facultad, quienes actuarán como nodos de supervisión para validar reportes y delegar las órdenes de trabajo al personal técnico.

RF - 07: Trazabilidad y Auditoría de Estados

Debe existir un panel de consulta que permita visualizar el flujo histórico de cada incidencia, detallando quién la reportó, quién la atendió, los cambios de estado y las pruebas de resolución.

RF - 08: Sistema de Privilegios por Roles (RBAC)

El acceso a las funciones del sistema estará restringido mediante un modelo de control de acceso basado en roles, diferenciando entre Reportador, Técnico de Mantenimiento y Administrador/Autoridad.

RF - 09: Catálogo de Unidades Académicas

El sistema debe proveer herramientas para la gestión (CRUD) de la información de las Facultades que integran la universidad.

1.2. Requisitos No Funcionales

RNF - 01: Optimización de la Experiencia de Usuario (UX)

La interfaz debe ser minimalista y fluida, reduciendo al mínimo la cantidad de clics necesarios para completar un reporte de incidencia.

RNF - 02: Eficiencia de Procesamiento

El software debe garantizar tiempos de respuesta inferiores a 2 segundos en la persistencia y consulta de datos bajo condiciones normales de carga.

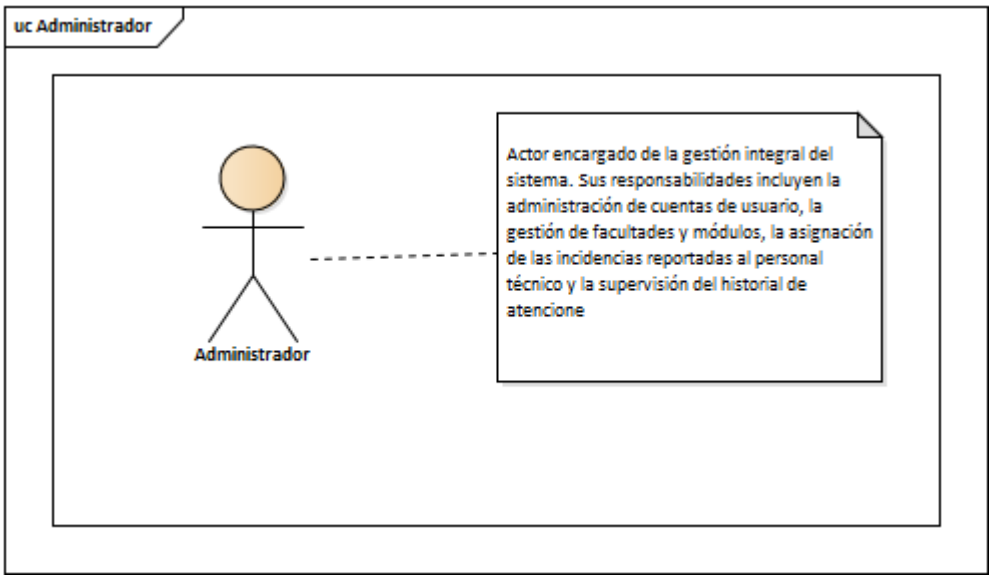
RNF - 03: Integridad y Seguridad de Datos

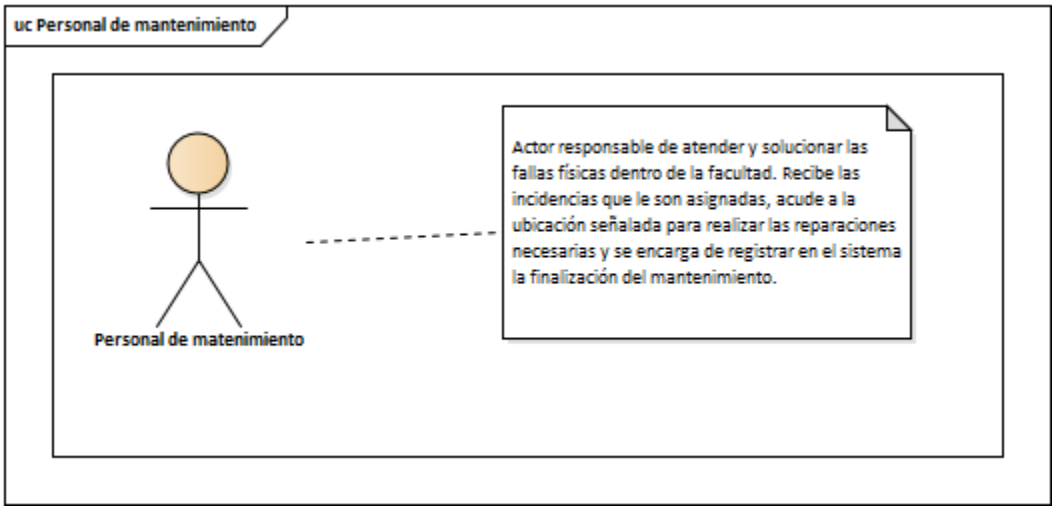
Es obligatorio el uso de protocolos de autenticación robustos (JWT) para asegurar que solo usuarios autorizados puedan interactuar con el backend, protegiendo la información contra accesos no permitidos.

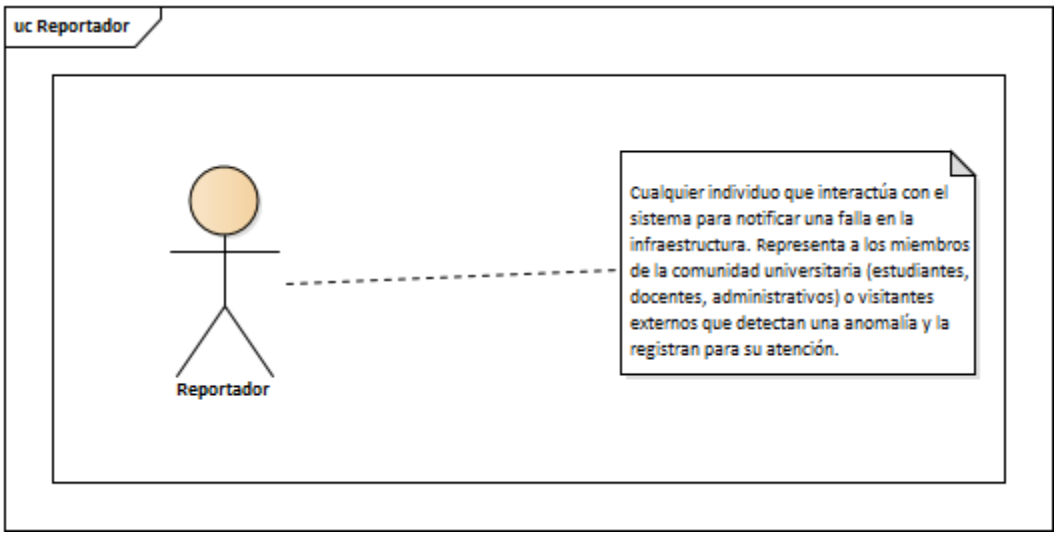
RNF - 04: Adaptabilidad Multiplataforma (Responsive Design)

Dado el alto uso de dispositivos móviles, la interfaz debe ser 100% responsiva, adaptándose automáticamente a diferentes resoluciones de pantalla sin perder funcionalidad.

1.3. Identificar Actores


uc Administrador
Administrador
Actor encargado de la gestión integral del sistema. Sus responsabilidades incluyen la administración de cuentas de usuario, la gestión de facultades y módulos, la asignación de las incidencias reportadas al personal técnico y la supervisión del historial de atencione


uc Personal de mantenimiento
Personal de matenimiento
Actor responsable de atender y solucionar las fallas físicas dentro de la facultad. Recibe las incidencias que le son asignadas, acude a la ubicación señalada para realizar las reparaciones necesarias y se encarga de registrar en el sistema la finalización del mantenimiento.



1.4. Identificar Casos de Uso

CU - 01: iniciar de sesión

CU - 02: Cerrar de sesión

CU - 03: Registro de los reportadores de incidencia

CU - 04: Gestión de usuarios

CU - 05: Gestionar infraestructura física

CU - 06: Registro de incidencia

CU - 07: Gestionar facultades

CU - 08: Registro de prueba de evidencia

CU - 09: Asignación de personal a incidencia

CU - 10: Visualizar incidencia

CU - 11: Finalizar orden de mantenimiento

CU - 12: Consulta de historial de incidencia

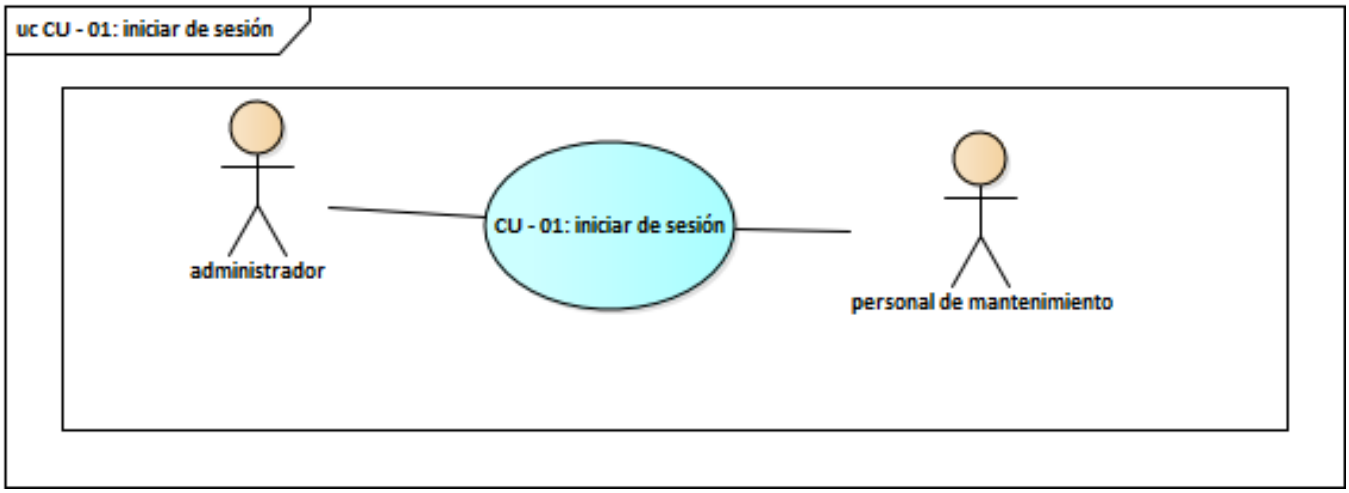
1.5. Priorizar Casos de Uso

ID	Nombre del Caso de Uso
CU - 01	iniciar de sesión
CU - 02	Cerrar de sesión
CU - 03	Registro de los reportadores de incidencia
CU - 04	Gestión de usuarios
CU - 05	Gestionar infraestructura física
CU - 06	Registro de incidencia
CU - 07	Gestionar facultades
CU - 08	Registro de prueba de evidencia
CU - 09	Asignación de personal a incidencia
CU - 10	Visualizar incidencia

ID	Nombre del Caso de Uso
CU - 11	Finalizar orden de mantenimiento
CU - 12	Consulta de historial de incidencia

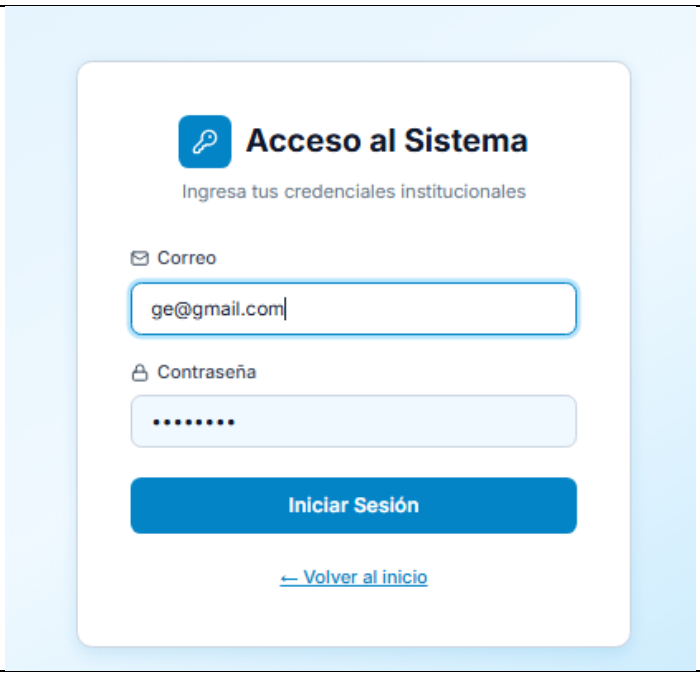
1.6. Detallar Casos de Uso

CU – 01: Inicio de sesión

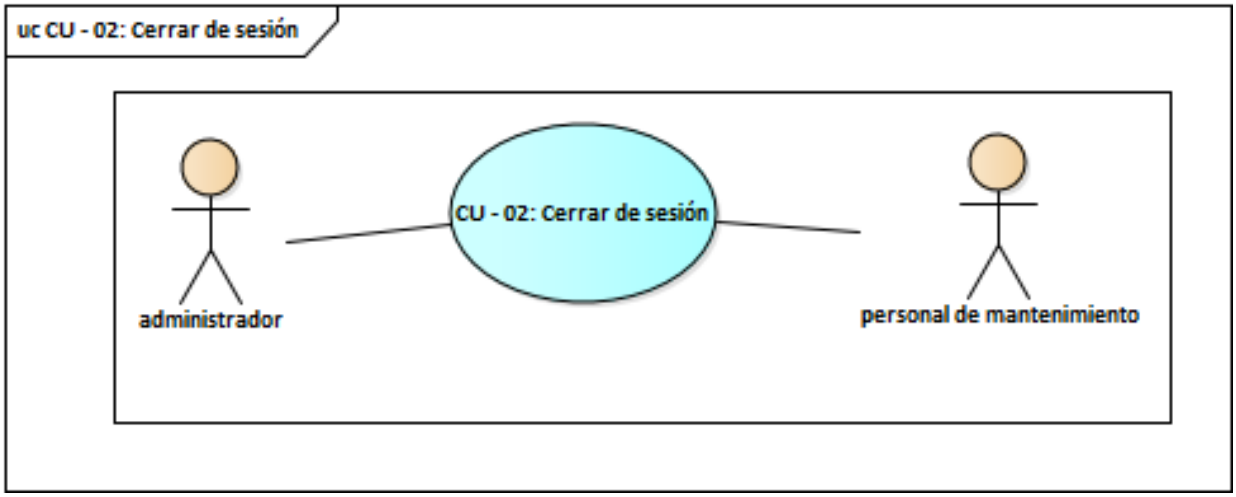


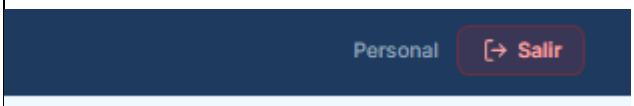
Campo	Descripción
Caso de Uso	CU-01; Iniciar de Sesión
Propósito	Brindar cuentas a los usuarios
Descripción	se va a brindar acceso al software a el decano y al encargado de mantenimiento

Actores	Administrador y Encargado de mantenimiento
Actor Iniciador	Administrador y Personal de mantenimiento
Precondición	Que los usuarios tengan cuenta, o sea que el decano y el encargado de mantenimiento estén registrados en el sistema
Proceso	**Administrador** 1. se escoge la opción correspondiente para entrar 2. pone sus credenciales correspondientes 3. se comprueba dicha credenciales, si son correctas tiene acceso al sistema **Encargado de mantenimeitno** 1. se escoge la opción correspondiente para entrar 2. pone sus credenciales correspondientes 3. se comprueba dicha credenciales, si son correctas tiene acceso al sistema
Postcondición	una vez verificadas las credenciales se les redirige a sus vistas correspondiente
Excepciones	Los espacios donde van a poner l<s credenciales deben tener sus respectivas en sus campos
Prototipo	

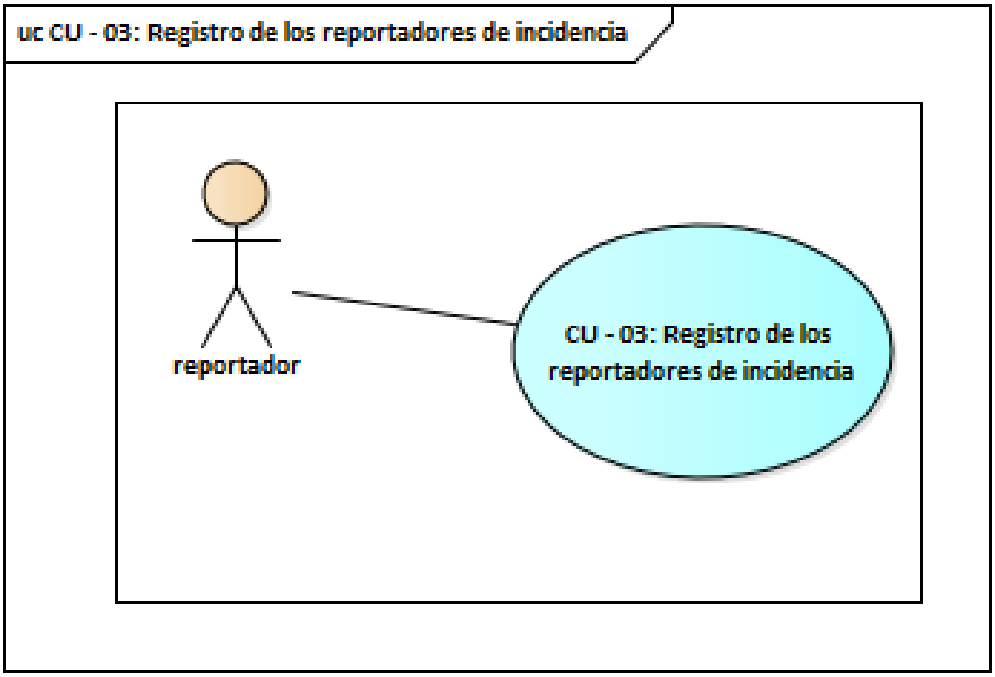


CU-02: Cierrar de sesión



Campo	Descripción
Caso de Uso	CU-02: Cierrar de Sesión
Propósito	Cerrar las sesiones iniciadas
Descripción	Se debe de poder cerrar las sesiones iniciadas por motivos de seguridad
Actores	Administrador y personal de mantenimiento
Actor Iniciador	Administrador y encargado de mantenimiento
Precondición	que se haya iniciado sesión dentro del software
Proceso	**Administrador y personal de mantenimiento** 1. se le da click al botón de cierre de sesión 2. se libera la sesión 3. el sistema al no tener sesion te redirige al home
Postcondición	Una vez se libera la sesión se le redirige al home
Excepciones	si al usuario se le expira la sesión se le libera directamente la sesión
Prototipo	

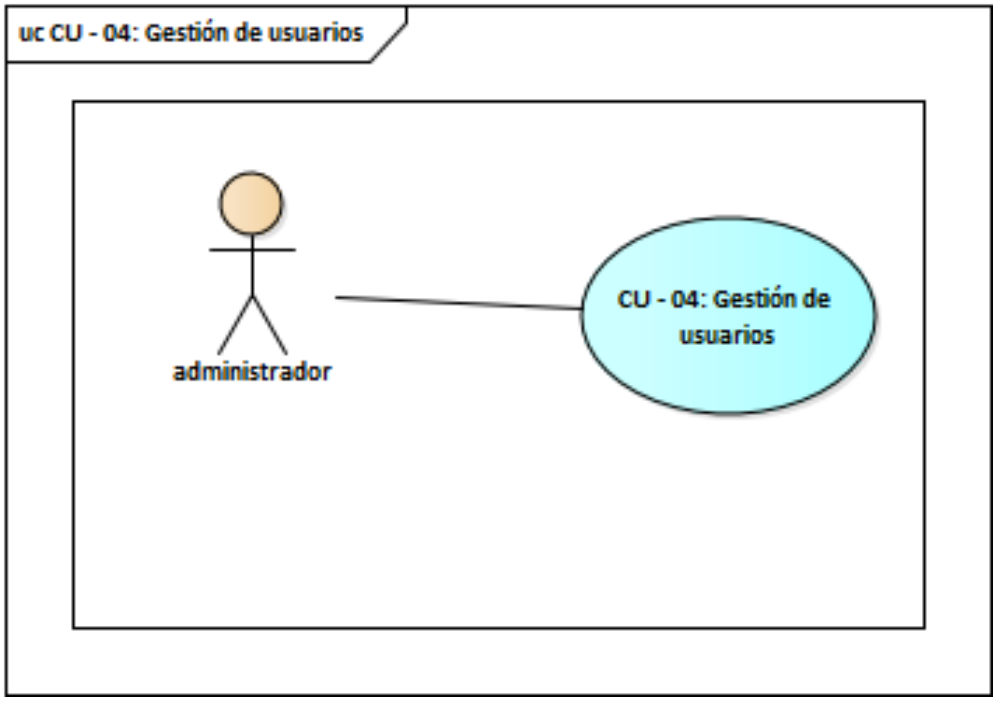
CU-03: Registro de los reportadores de incidencia



Campo	Descripción
Caso de Uso	CU-03: Registro de los reportadores de incidencia
Propósito	Tener registro de los que reporten problemas dentro de la facultad
Descripción	Se debe de tener los registro de los que reportan las incidencias que ocurran dentro de la facultad
Actores	Reportador
Actor Iniciador	Reportador

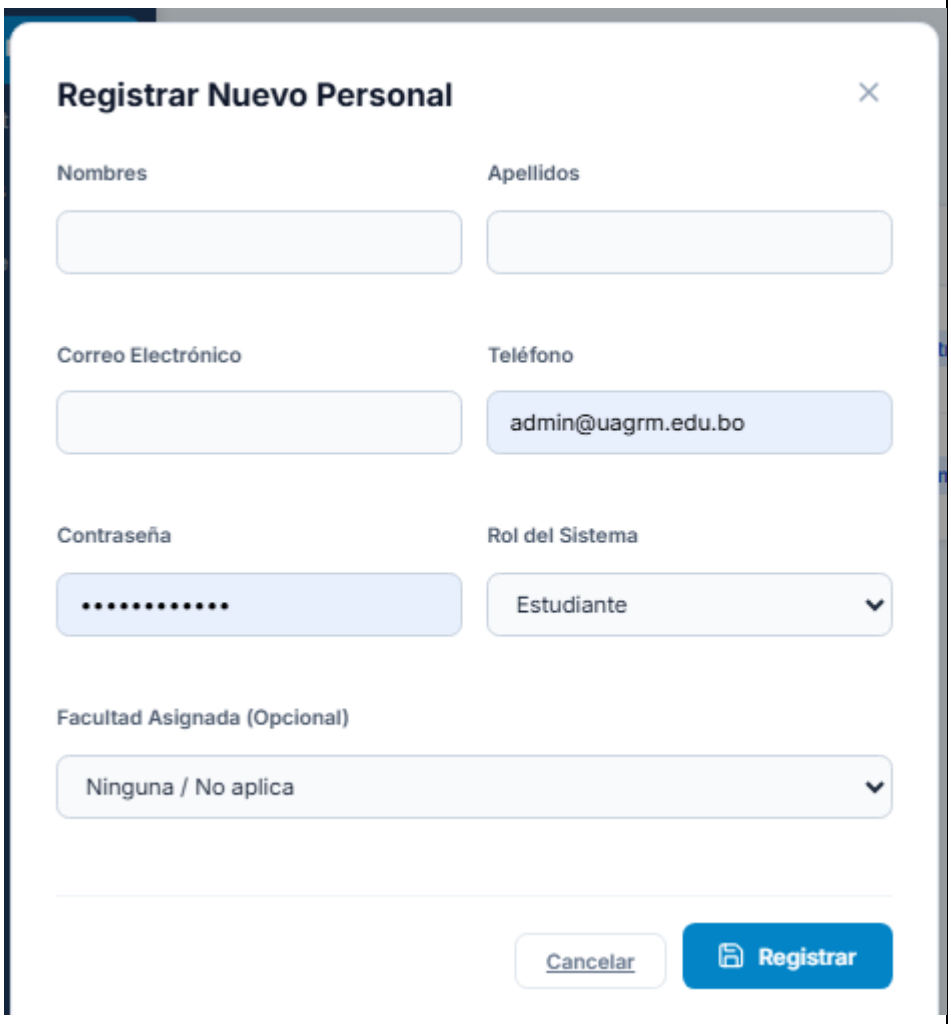
Campo	Descripción
Precondición	que se quiera reportar un problema
Proceso	1. se llenan los datos del formulario a la hora de reportar una incidencia 2. se envían los datos y se registran
Postcondición	los datos se registran exitosamente
Excepciones	• se verifica que los espacios no esten vacios • se verifica que no haya caracteres incorrectos en los campos incorrectos, como ser números dentro del nombre o letras dentro del número del celular
Prototipo	

CU-04:Gestion de usuarios

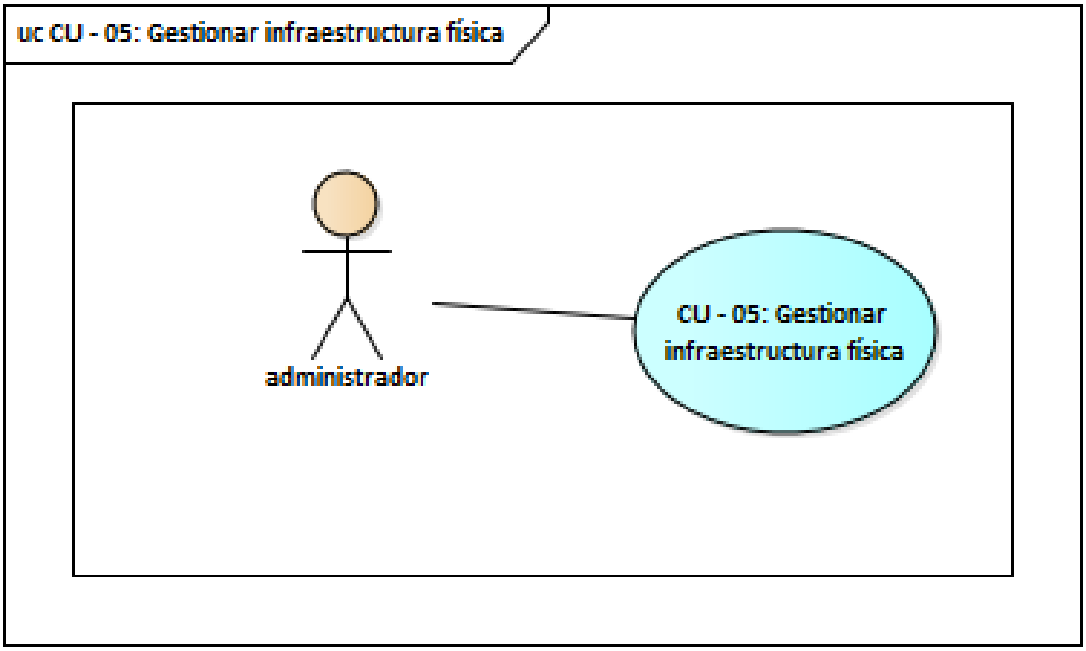


Campo	Descripción
Caso de Uso	CU - 04: Gestión de usuarios
Propósito	Administrar las cuentas de acceso para Administrador y Encargados de Mantenimiento
Descripción	se deberá poder administrar las cuentas que tengan acceso al software
Actores	Administrador

Campo	Descripción
Actor Iniciador	Administrador
Precondición	se tiene que iniciar sesión
Proceso	**Registro** 1. Se loguea 2. Se va a la vista de usuarios 3. Se le da al botón de registro 4. Se rellena el formulario 5. Se registra al usuario **Actualización** 1. Se loguea 2. Se va a la vista de usuarios 3. Se le da al botón de actualizar al usuario correspondiente 4. Se rellena el formulario 5. Se actualiza al usuario escogido **Eliminar** 1. Se loguea 2. Se va a la vista de usuarios 3. Se le da al botón de eliminar al usuario correspondiente 4. Se elimina al usuario escogido
Postcondición	Se avisara si el proceso fue exitoso o no al usuario del registro, actualizacion y eliminacion

Campo	Descripción
Excepciones	los formularios de registro y actualizacion van a verificar los datos, osea que no se mande datos incorrectos
prototipo	

CU-05:gestionar infraestructura física

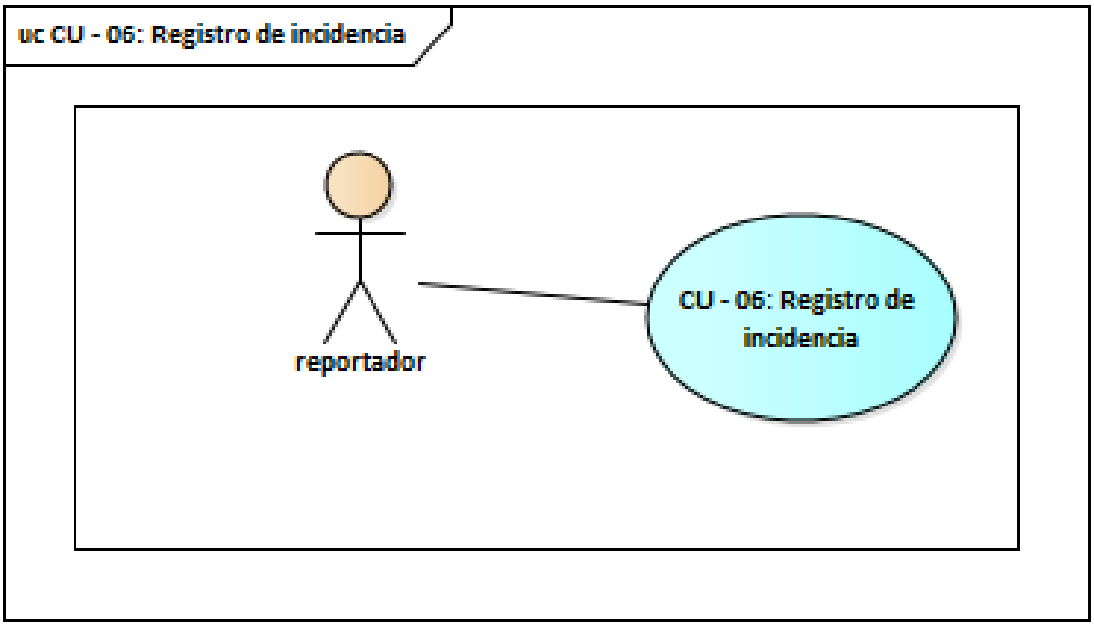


Campo	Descripción
Caso de Uso	CU-05: Gestionar infraestructura física
Propósito	Mantener actualizado el mapa físico de la universidad para reportes precisos
Descripción	se mantendrá un mapa de las facultades, de cuántos módulos tiene y cuales son sus características, ejemplo: las aulas áreas de descanso , etc
Actores	Administrador
Actor Iniciador	Administrador
Precondición	loguearse y debe existir al menos una Facultad registrada
Proceso	**Registro**

Campo	Descripción
	1. loguearse 2. ir a la vista de gestión de estructura física 3. darle al botón de registrarse 4. llenar el formulario 5. registrar la estructura física **Actualización** 1. loguearse 2. ir a la vista de gestión de estructura física 3. darle al botón de actualizar la estructura correspondiente 4. llenar el formulario 5. se actualiza la estructura correspondiente **Eliminación** 1. loguearse 2. ir a la vista de gestión de estructuras físicas 3. darle al botón de eliminar la estructura correspondiente 4. se elimina la estructura correspondiente
Postcondición	se notificara si el registro,actualización o eliminación fue exitoso o no

Campo	Descripción
Excepciones	los formularios del registro y actualización van a estar validados contra espacios en blanco, o datos incorrectos
Prototipo	

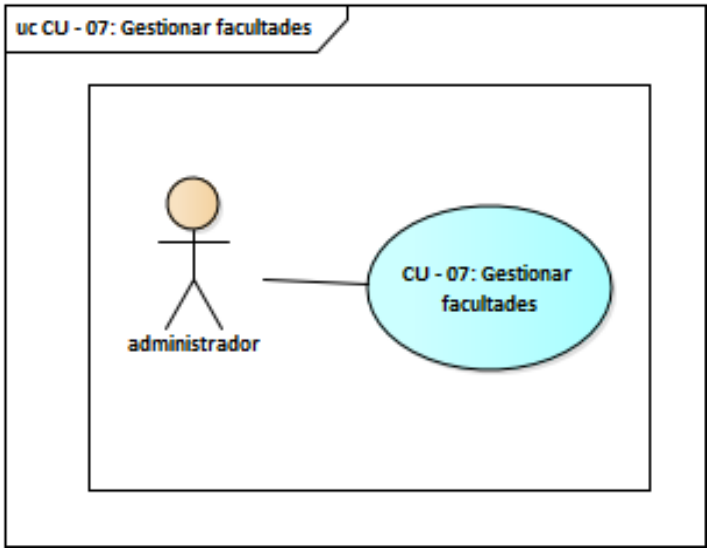
CU-06:registro de incidencia



Campo	Descripción
Caso de Uso	CU - 06: Registro de notas de problemas
Propósito	Permitir que los usuarios reporten una o más incidencias encontradas en un mismo ambiente
Descripción	los problemas encontrados por estudiantes, docente u otro, se van a registrar para tener constancia de estos
Actores	Reportador
Actor Iniciador	Reportador

Campo	Descripción
Precondición	entrar al sitio web, no se necesita tener cuenta para esto
Proceso	**Reportador de problema** 1. entrar al sitio web 2. darle a registrar el problema o problemas que encontró 3. llenar el formulario 4. enviar el problema
Postcondición	se avisa del envío correcto del problema
Excepciones	los formularios validaran los datos correctamente
prototipo	

CU-07: gestionar facultades

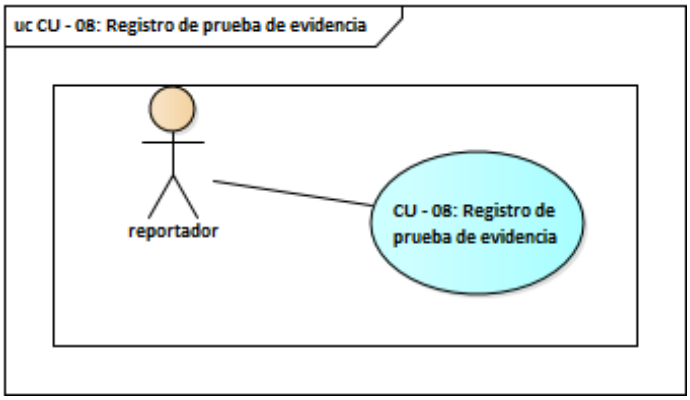


Campo	Descripción
Caso de Uso	CU-07: Gestionar Facultades
Propósito	Gestionar las facultades que van a estar en el software
Descripción	se necesita un registro de las facultades que existen en la universidad
Actores	Administrador
Actor Iniciador	Administrador
Precondición	loguearse

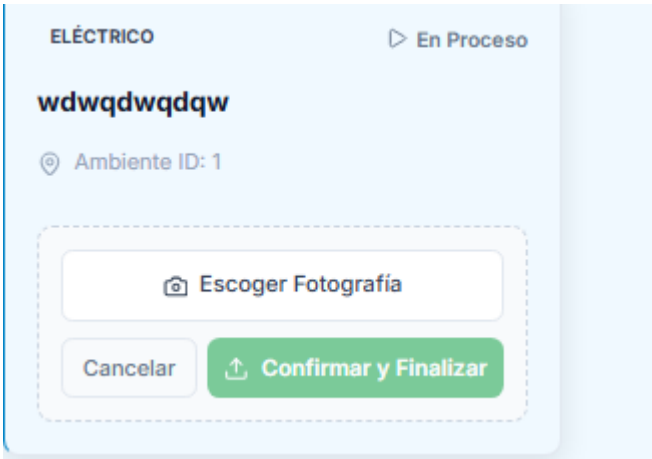
Campo	Descripción
Proceso	**Registro** 1. se loguea 2. se va a la vista de gestión de facultades 3. se le da al botón de registrar 4. se llena el formulario 5. se registra **Actualización** 1. se loguea 2. se va a la vista de gestión de facultades 3. se le da al botón de actualizar la facultad correspondiente 4. se llena el formulario 5. se actualiza la facultad correspondiente
Postcondición	se avisara si el registro o la actualización fue exitosa o no
Excepciones	se validara los campos requeridos en el formulario

Campo	Descripción
prototipo	

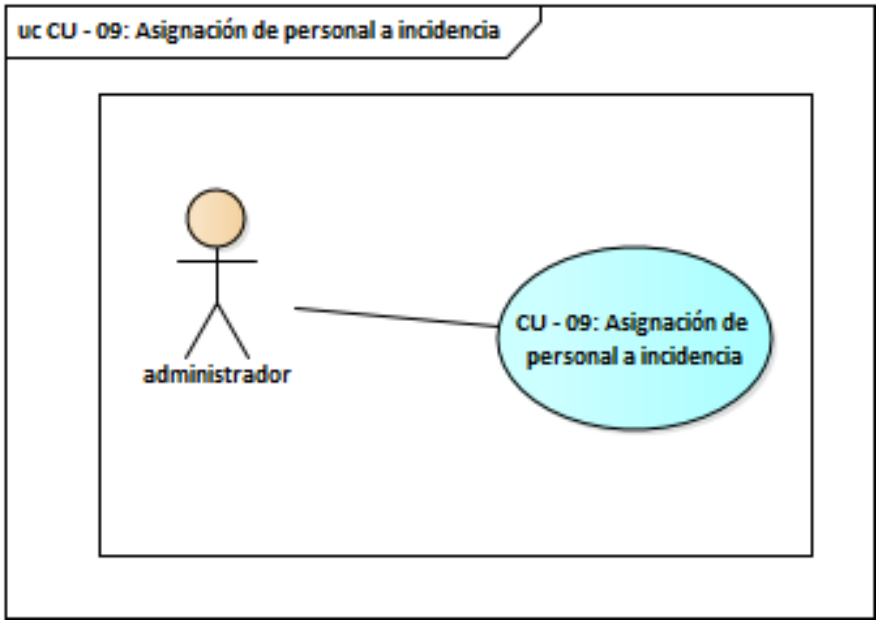
CU08:registro de prueba de evidencia



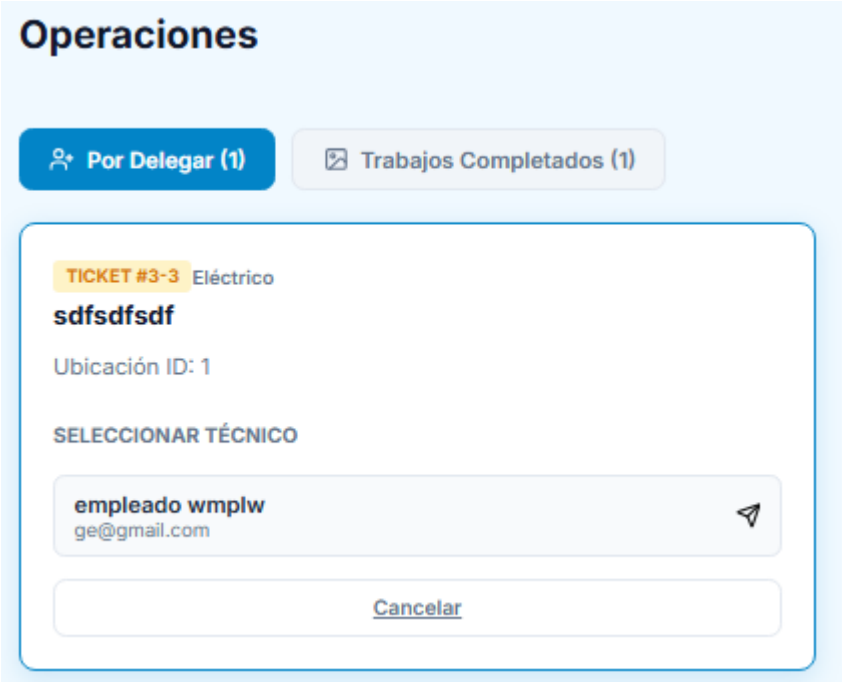
Campo	Descripción
Caso de Uso	CU - 08: Registro de prueba de evidencia
Propósito	Proporcionar material fotográfico que pruebe la existencia o la reparación de una falla
Descripción	se va a registrar fotografías del problema si es que es necesario (es opcional)
Actores	encargado de mantenimiento
Actor Iniciador	Reportador del problema, encargado de mantenimiento
Precondición	estar llenando el formulario de registro de problema
Proceso	1. se entra a la web 2. se le da al botón de registrar el problema 3. se sube las foto del problema
Postcondición	al darle al botón de registrar problema si el reportador de problemas subió una foto se guarda si no, se guarda sin foto

Campo	Descripción
Excepciones	

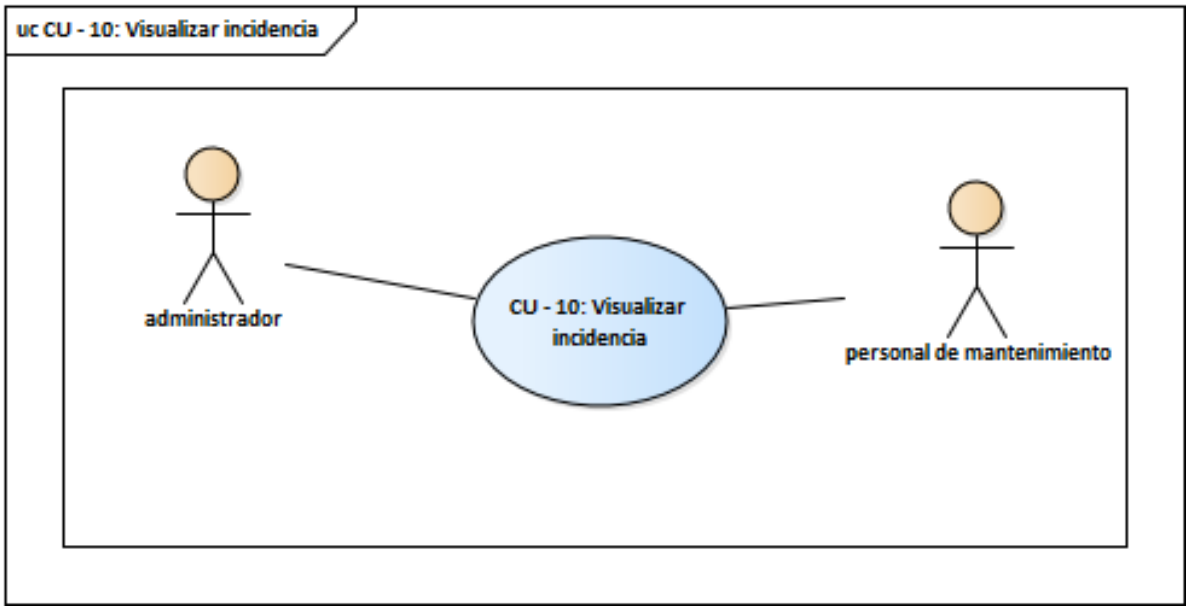
CU09:asignación de personal a incidencia



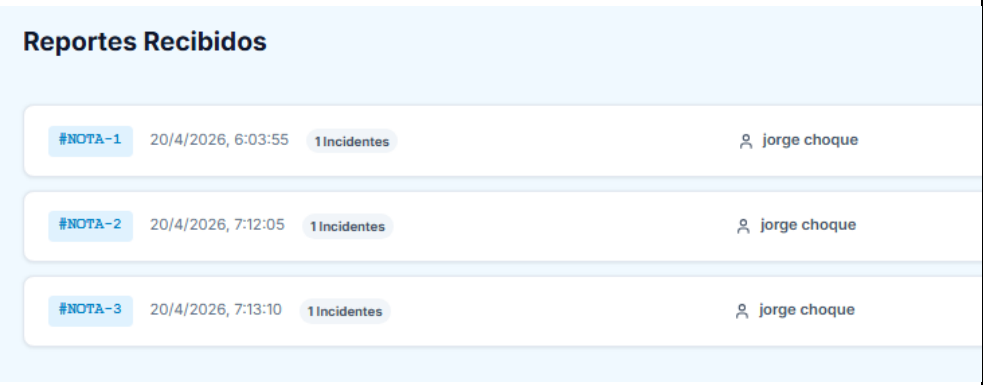
Campo	Descripción
Caso de Uso	CU - 09 Asignación de personal a incidencia
Propósito	se necesita asignar a un encargado de mantenimiento a un problema
Descripción	se va a asignar problemas a resolver a un encargado de mantenimeinto
Actores	Administrador
Actor Iniciador	Administrador
Precondición	estar logueado
Proceso	**Registro** 1. se loguea 2. se va a la vista de asignación 3. se le da al botón de asignar problema 4. se asocia un problema a un encargado de mantenimiento **Actualizar** 1. se loguea 2. se va a la vista de asignación 3. se escoge una asignación 4. se le da a actualizar asignación y se cambia al encargado de mantenimiento

Campo	Descripción
Postcondición	se avisa de si la asignación tu éxito o no
Excepciones	que el encargado exista, que el problema exista
Prototipo	

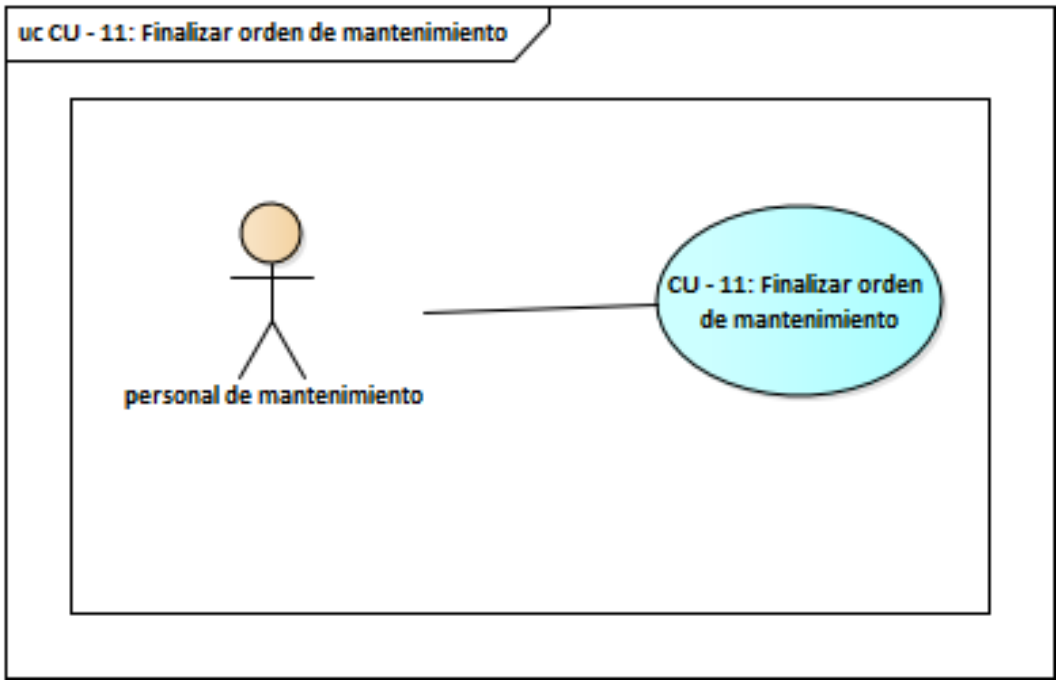
CU10:visualizar incidencia



Campo	Descripción
Caso de Uso	CU - 10: Visualizar incidencia
Propósito	visualizar los problemas que haya
Descripción	el decano y los encargados de mantenimiento deben de poder visualizar los problemas que haya
Actores	Administrador , personal de mantenimiento

Campo	Descripción
Actor Iniciador	Administrador , personal de mantenimiento
Precondición	estar logueado
Proceso	Administrador y el encargado de mantenimiento 1. se logra 2. se va a la vista de problemas y se puede ver
Postcondición	ninguna
Excepciones	ninguna
PROTOTIPO	

CU11:finalizar orden de mantenimiento

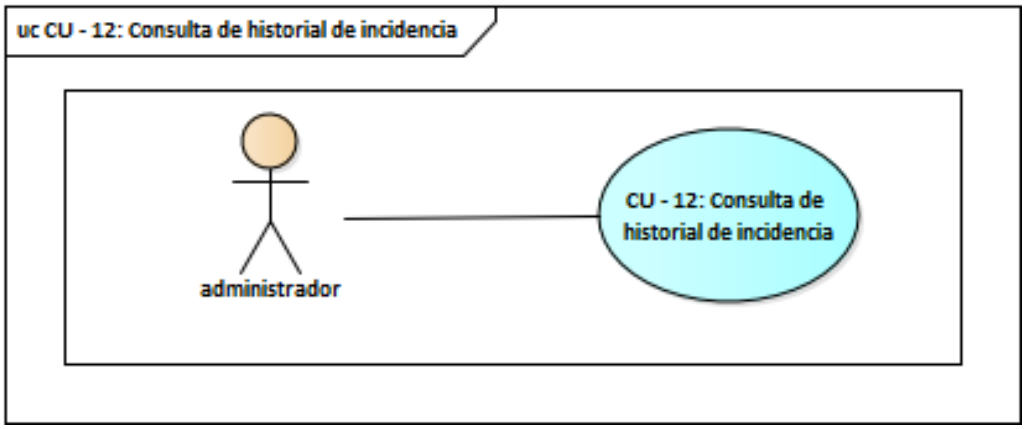


Campo	Descripción
Caso de Uso	CU - 11: Finalizar orden de mantenimiento
Propósito	que se pueda dar finalización al mantenimiento
Descripción	el software debe de poder finalizar el mantenimiento
Actores	personal de mantenimiento
Actor Iniciador	personal de mantenimiento

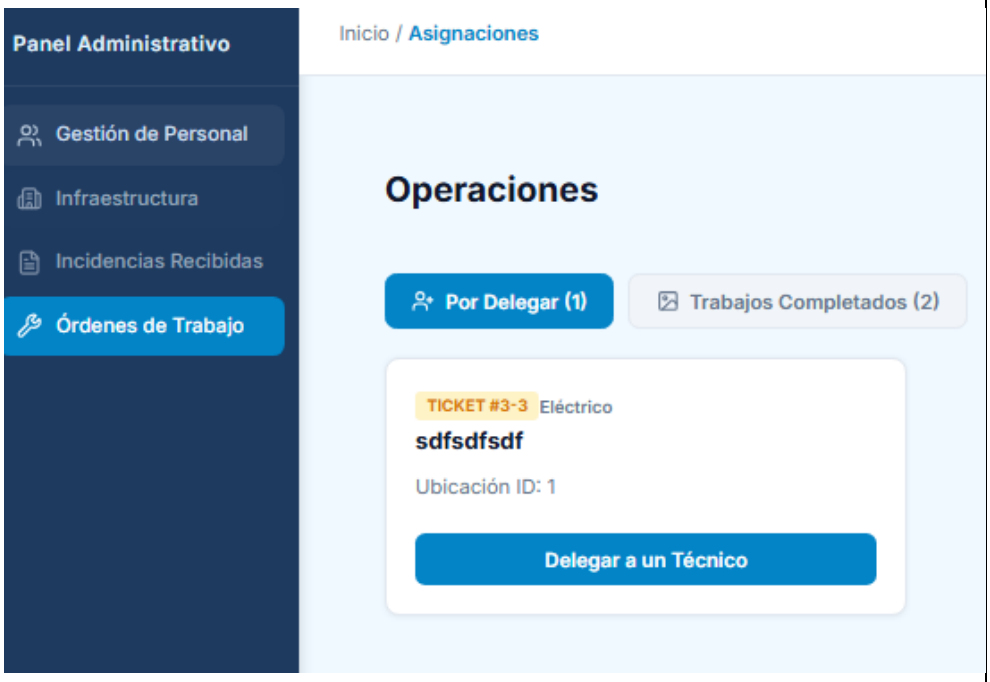
Campo	Descripción
Precondición	estar logueado
Proceso	1. se logra 2. se va a la vista de problemas y se cambia el estado a finalización de mantenimiento
Postcondición	ninguna
Excepciones	ninguna

Campo	Descripción
PROTOTIPO	

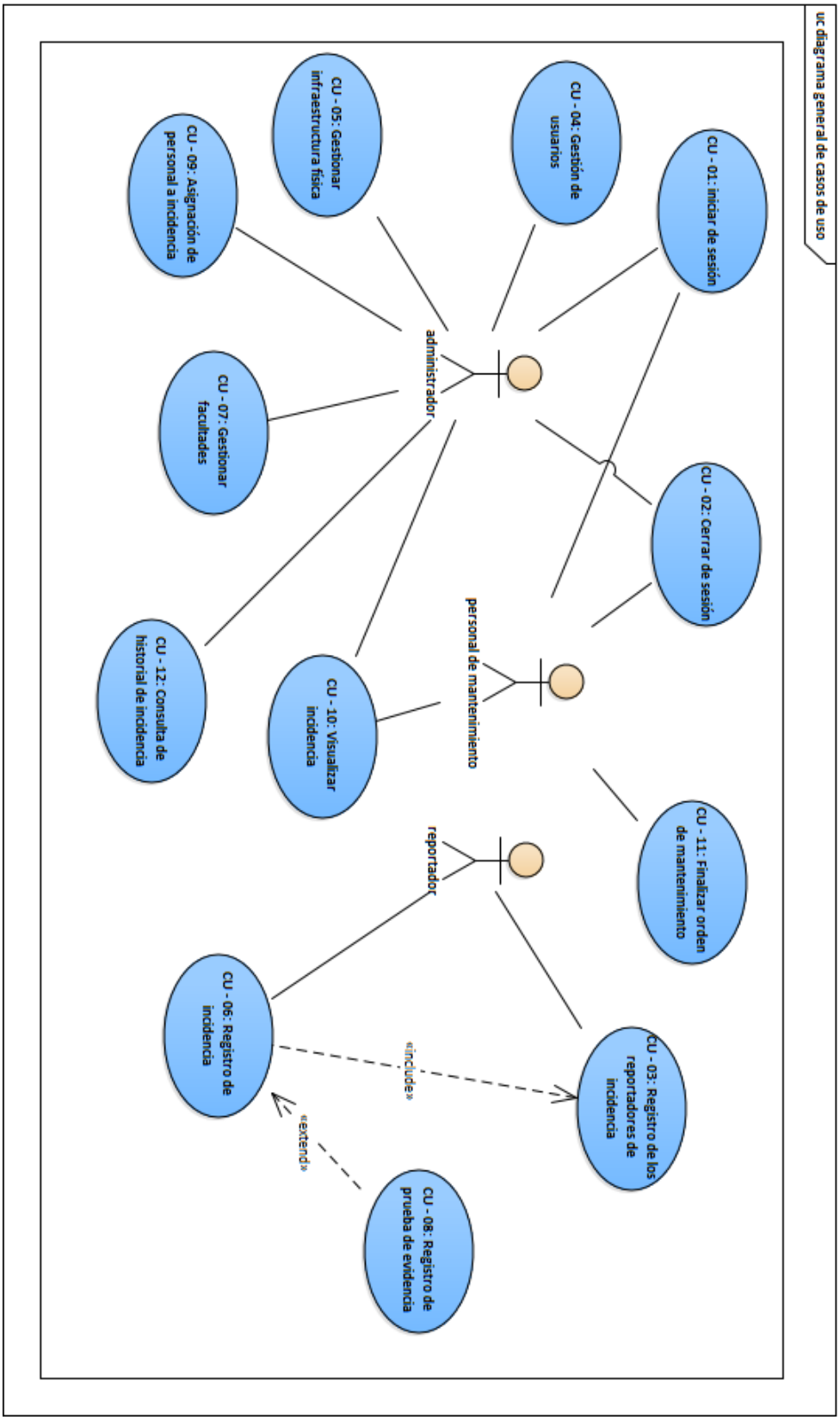
CU12:consulta de historial de incidencia



Campo	Descripción
Caso de Uso	CU - 12: Consulta de historial de incidencia
Propósito	Visualizar la cronología de estados y acciones realizadas sobre un problema específico
Descripción	se podrá hacer consultas al historial de atención
Actores	Administrador
Actor Iniciador	Administrador
Precondición	El problema debe tener al menos un cambio de estado registrado

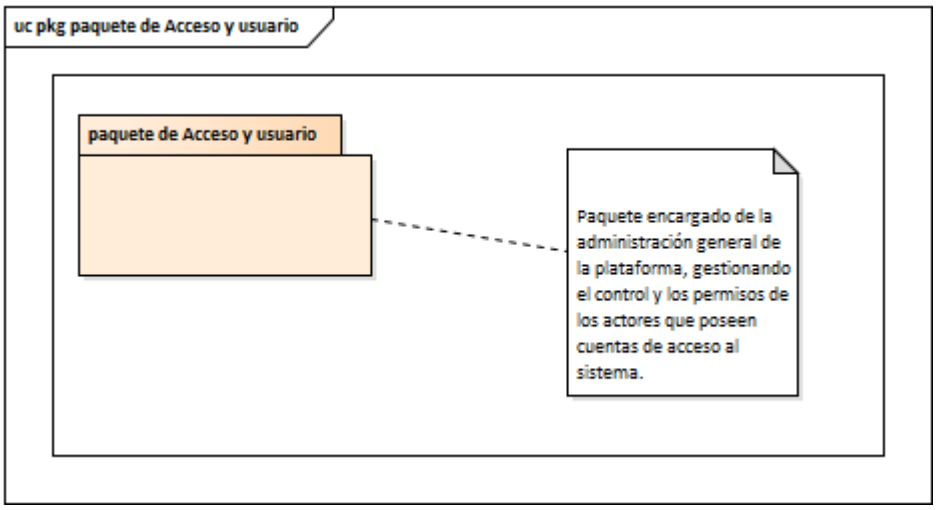
Campo	Descripción
Proceso	• se entra a la pestaña de asignaciones • , va aver una parte de las problemas que ya se resolvieron • es más que todo para poder visualizar estos problemas
Postcondición	El usuario obtiene una visión clara del proceso de resolución y los tiempos de respuesta
Excepciones	ninguna
PROTOTIPO	

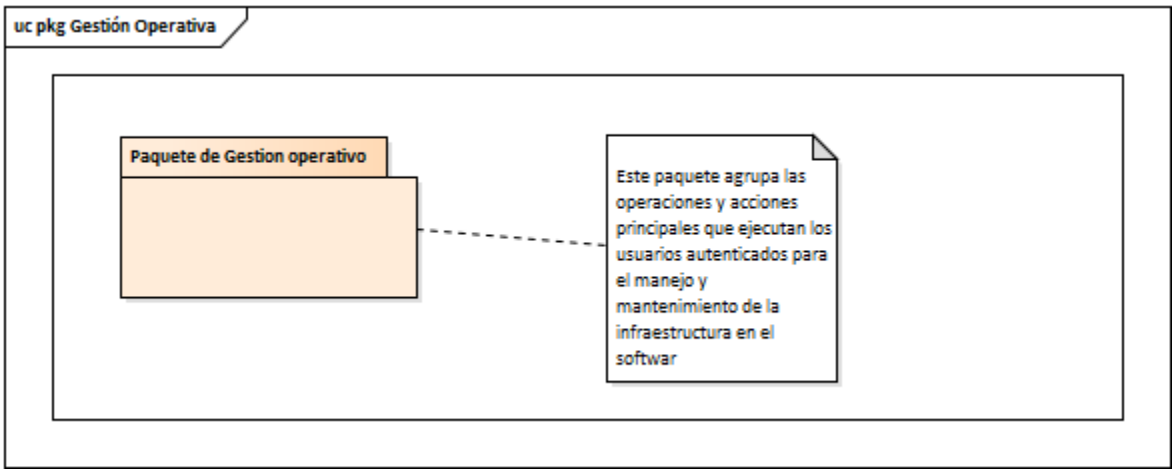
1.7. Diagrama general de Casos de Uso

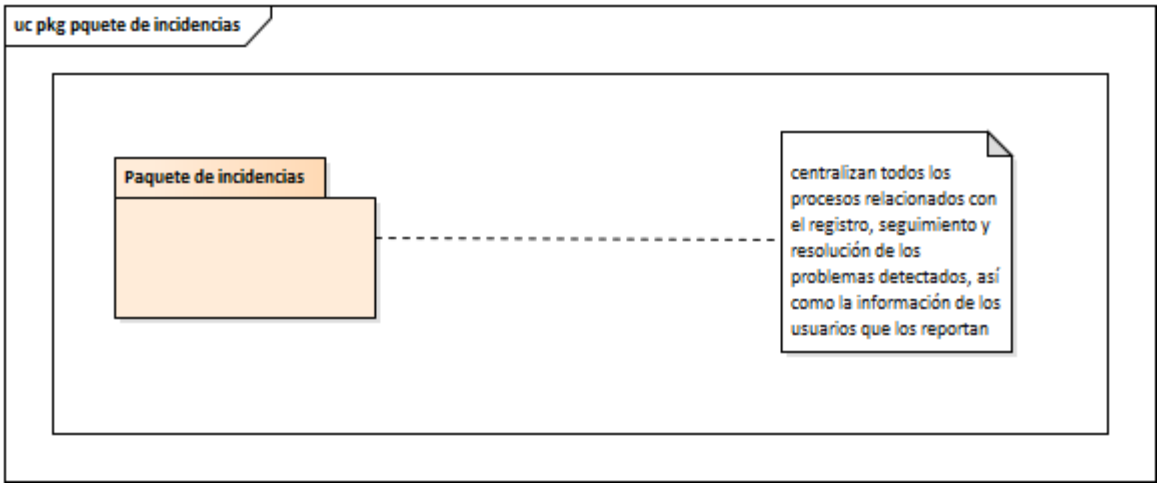


CAPITULO 2: ANALISIS

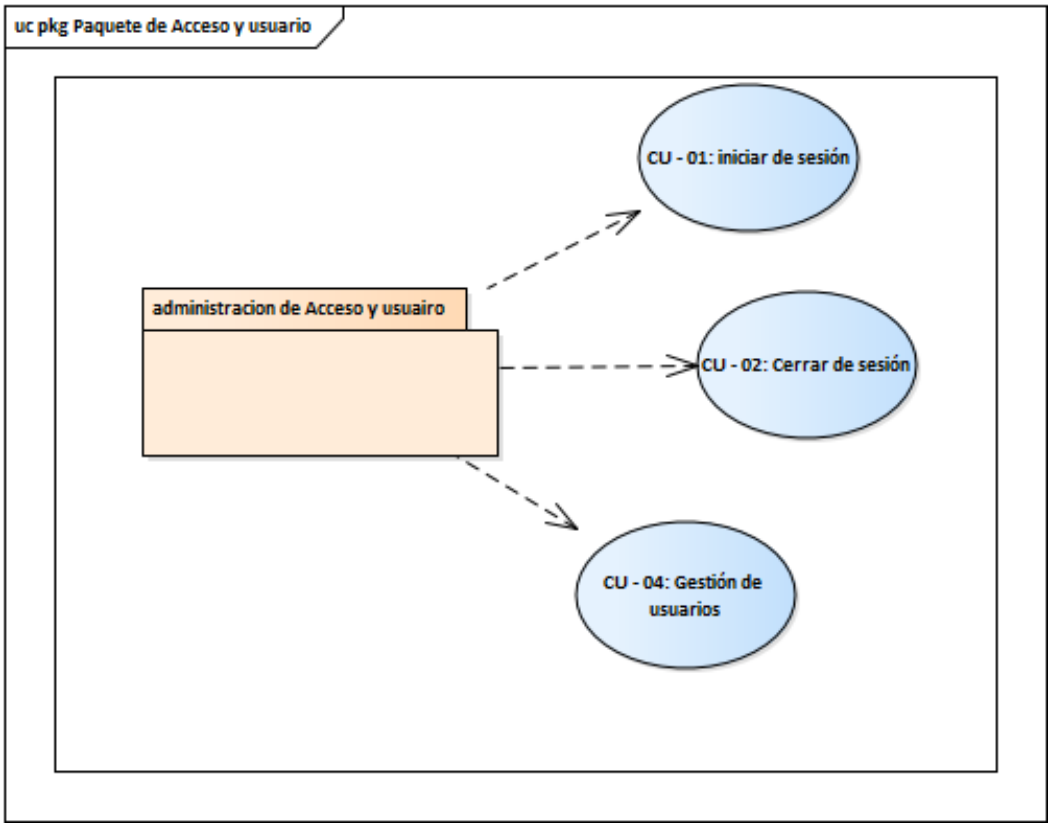
2.1. identificación de Paquetes

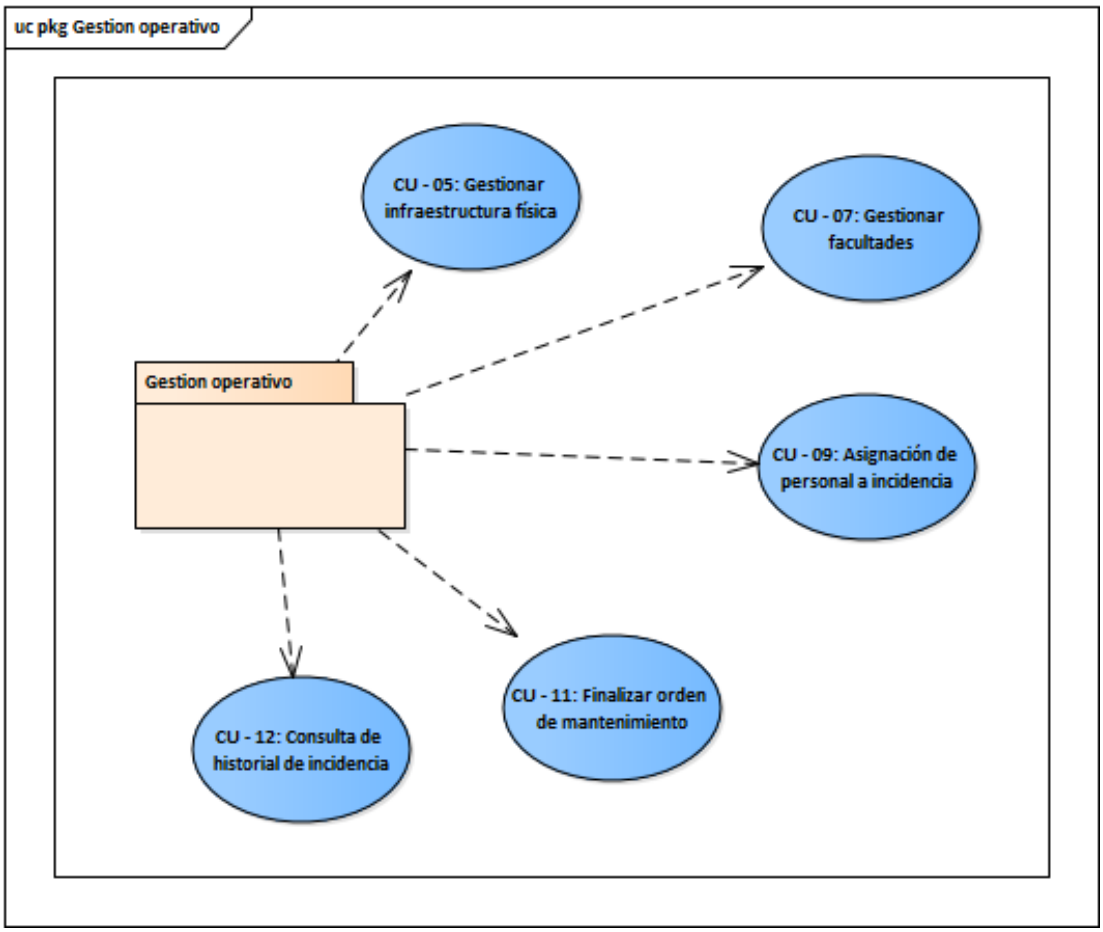


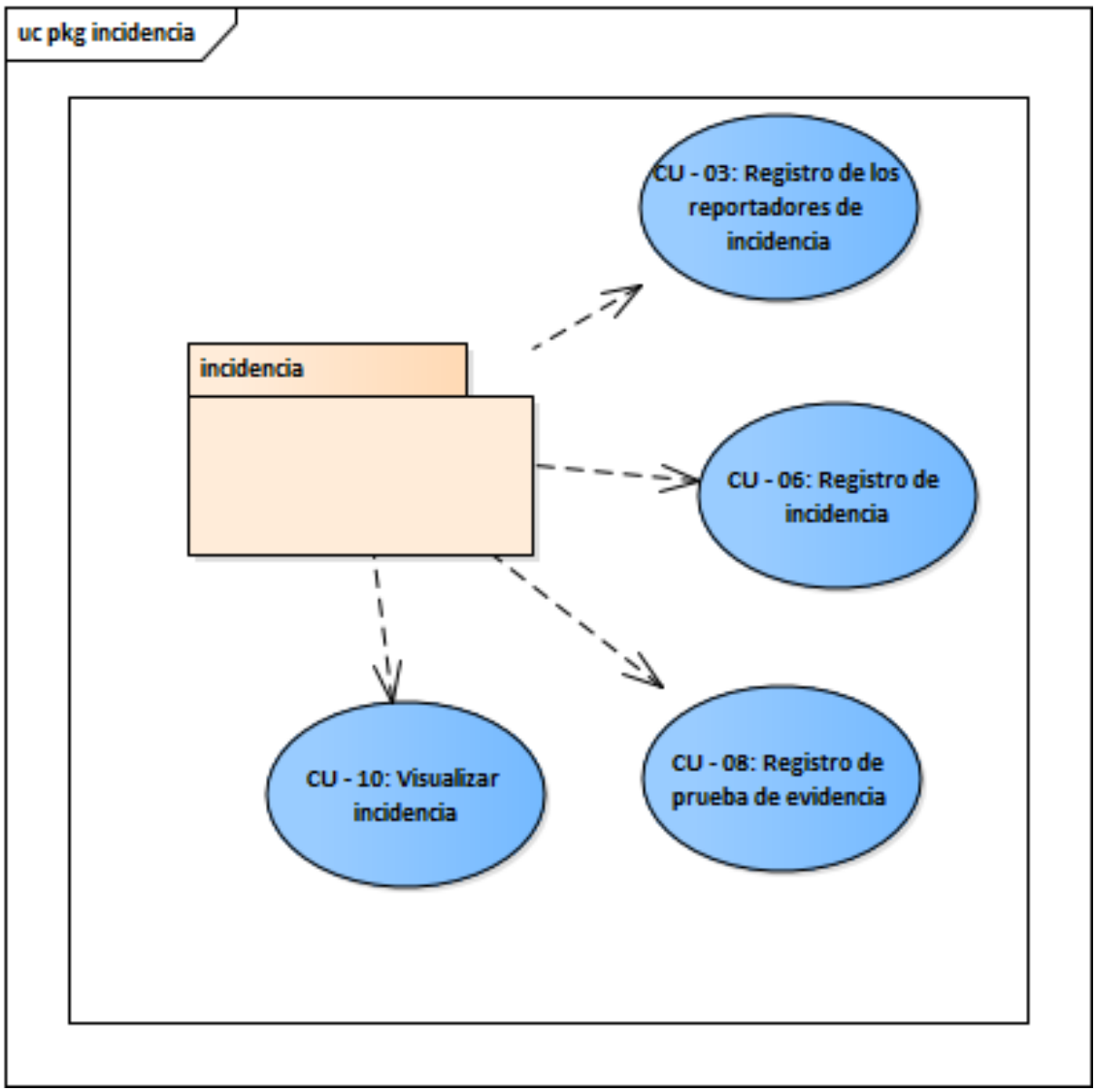




2.2. relacionar paquetes con Casos de Uso

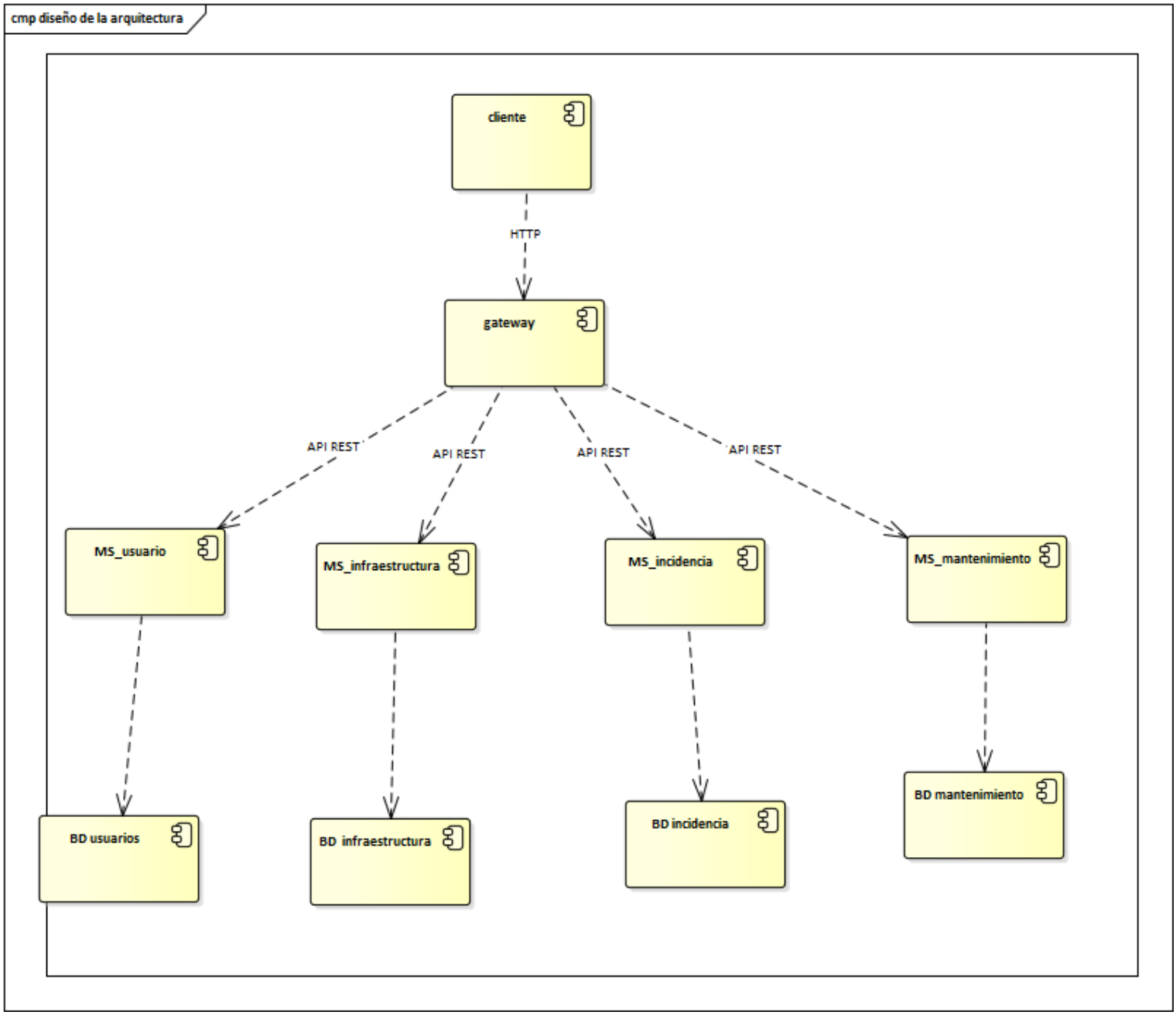



uc pkg Gestion operativo
CU - 05: Gestionar infraestructura física
CU - 07: Gestionar facultades
Gestion operativo
CU - 09: Asignación de personal a incidencia
CU - 11: Finalizar orden de mantenimiento
CU - 12: Consulta de historial de incidencia


uc pkg incidencia
incidencia
CU - 03: Registro de los reportadores de incidencia
CU - 06: Registro de incidencia
CU - 10: Visualizar incidencia
CU - 08: Registro de prueba de evidencia

CAPTULO 3: DISEÑO

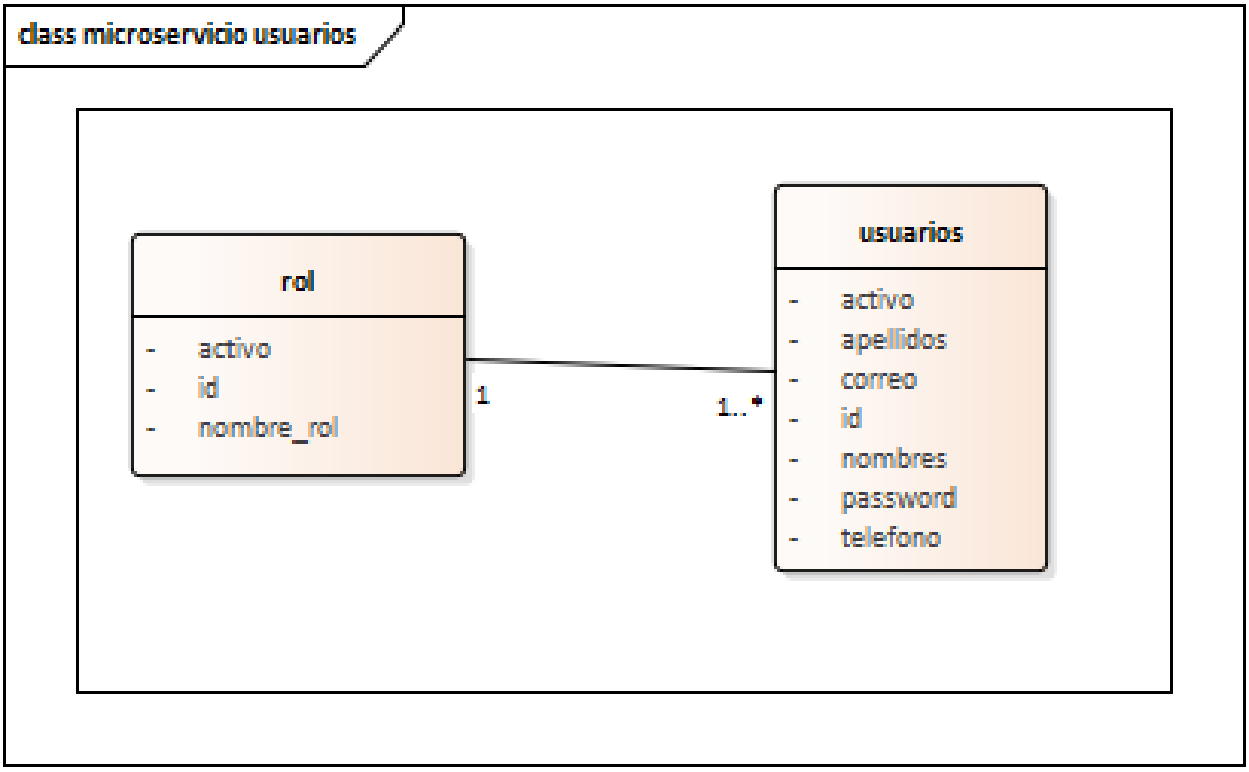
3.1. Diseño de la Arquitectura



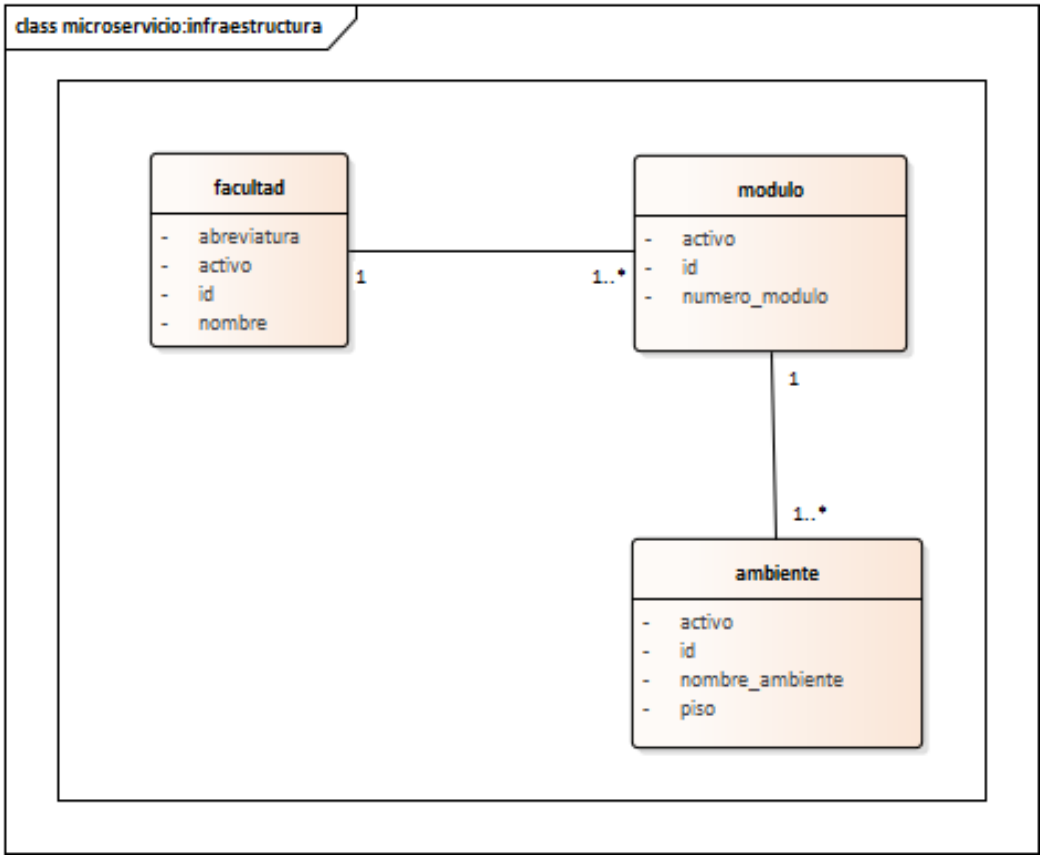
3.2. Diseño de la Base de Datos

3.2.1. Diseño conceptual

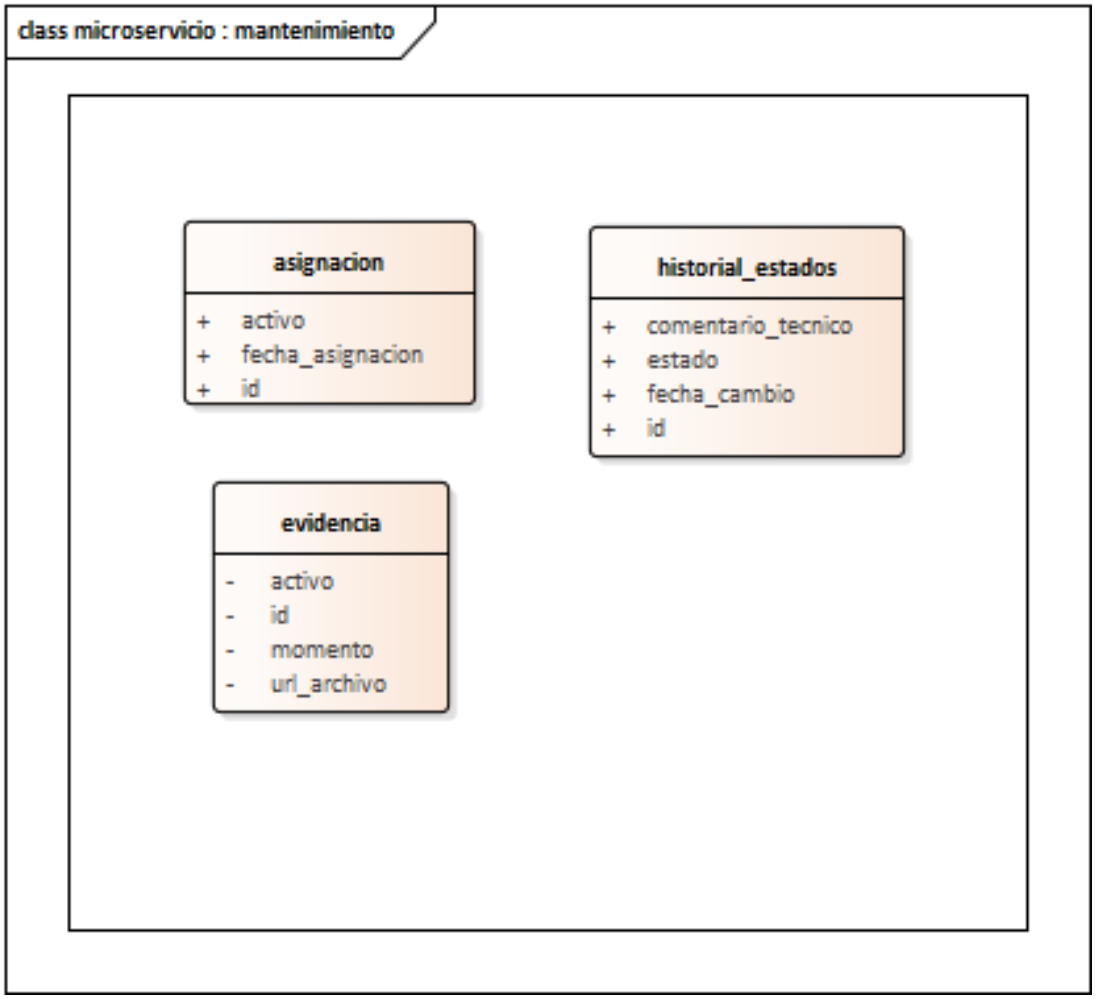
Usuarios



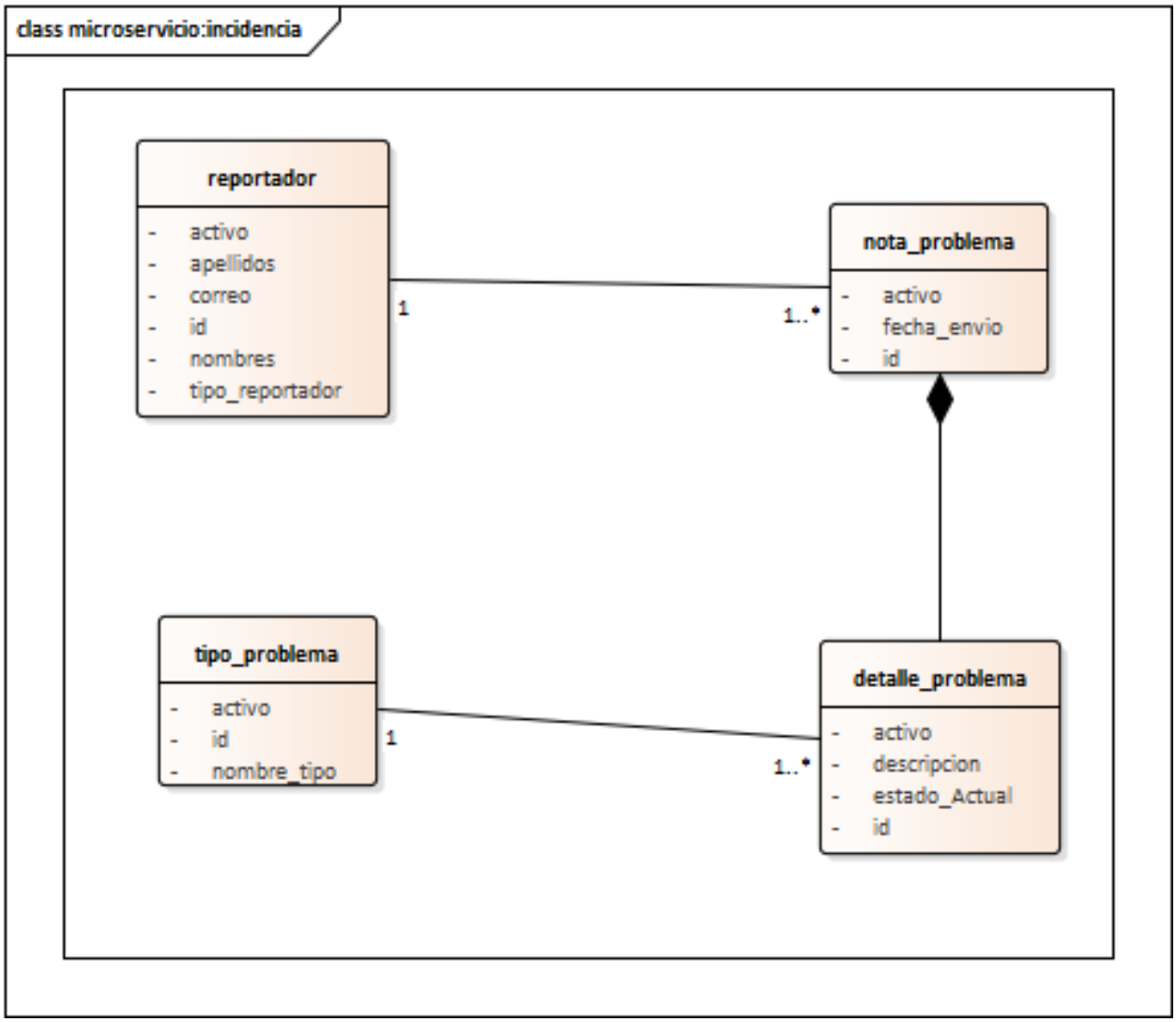
Infraestructura



Mantenimiento



Incidencia


class microservicio:incidencia
reportador
- activo
- apellidos
- correo
- id
- nombres
- tipo_reportador
1
1..*
nota_problema
- activo
- fecha_envio
- id
tipo_problema
- activo
- id
- nombre_tipo
1
1..*
detalle_problema
- activo
- descripcion
- estado_Actual
- id

3.2.2. Diseño Logico

Usuarios

rol

PK	
id	nombre_rol

usuarios

PK						FK		
id	nombres	apellidos	telefono	correo	password	rol_id	facultad_id	activo

Infraestructura

facultad

PK			
id	nombre	abreviatura	activo

ambiente

PK			FK	
id	nombre_ambiente	piso	modulo_id	activo

modulo

PK		FK	
id	nombre_modulo	facultad_id	activo

MS:Incidencia

reportador

PK					
id	nombres	apellidos	correo	tipo_reportador	activo

tipo_incidencia

PK		
id	nombre_tipo	activo

nota_problemas

PK		FK	
id	fecha_envio	reportador_id	activo

detalle_problema

PK				FK	FK	
id	descripcion	estado_actual	ambiente_id	nota_id	tipo_incidencia_id	activo

MS_Mantenimiento

asignacion

PK				
id	fecha_asignacion	detalle_problema_id	usuario_id	activo

historial_Estados

PK					FK	
id	tipo	estado	fecha_cambio	comentario_tecnico	asignacion_id	activo

evidencia

PK			FK	
id	url_archivo	momento	asignacion_id	activo

3.2.3. Diseño Fisico

Usuarios

```
CREATE TABLE IF NOT EXISTS "rol" (
   "id" SERIAL PRIMARY KEY,
   "nombre_rol" VARCHAR(50) NOT NULL,
   "activo" BOOLEAN DEFAULT TRUE
);

CREATE TABLE IF NOT EXISTS "usuarios" (
   "id" SERIAL PRIMARY KEY,
   "nombres" VARCHAR(100) NOT NULL,
   "apellidos" VARCHAR(100) NOT NULL,
   "telefono" VARCHAR(20),
   "correo" VARCHAR(100) UNIQUE NOT NULL,
   "password" VARCHAR(255) NOT NULL,
   "rol_id" INTEGER REFERENCES "rol"("id") ON DELETE SET NULL,
   "facultad_id" INTEGER,
   "activo" BOOLEAN DEFAULT TRUE
);
```

Infraestructura

```
CREATE TABLE IF NOT EXISTS "facultad" (
   "id" SERIAL PRIMARY KEY,
   "nombre" VARCHAR(150) NOT NULL,
   "abreviatura" VARCHAR(10),
```

```
    "activo" BOOLEAN DEFAULT TRUE
);

CREATE TABLE IF NOT EXISTS "modulo" (
    "id" SERIAL PRIMARY KEY,
    "numero_modulo" VARCHAR(20) NOT NULL,
    "facultad_id" INTEGER REFERENCES "facultad"("id") ON DELETE CASCADE,
    "activo" BOOLEAN DEFAULT TRUE
);

CREATE TABLE IF NOT EXISTS "ambiente" (
    "id" SERIAL PRIMARY KEY,
    "nombre_ambiente" VARCHAR(100) NOT NULL,
    "piso" INTEGER,
    "modulo_id" INTEGER REFERENCES "modulo"("id") ON DELETE CASCADE,
    "activo" BOOLEAN DEFAULT TRUE
);
```

Incidencia

```
CREATE TABLE IF NOT EXISTS "reportador" (
    "id" SERIAL PRIMARY KEY,
    "nombres" VARCHAR(100) NOT NULL,
    "apellidos" VARCHAR(100) NOT NULL,
    "correo" VARCHAR(100) NOT NULL,
    "tipo_reportador" VARCHAR(50) NOT NULL, -- Estudiante, Docente, Administrativo
    "activo" BOOLEAN DEFAULT TRUE
);
```

```
CREATE TABLE IF NOT EXISTS "tipo_incidencia" (
   "id" SERIAL PRIMARY KEY,
   "nombre_tipo" VARCHAR(100) NOT NULL, -- Eléctrico, Mobiliario, Paredes, etc.
   "activo" BOOLEAN DEFAULT TRUE
);

CREATE TABLE IF NOT EXISTS "nota_problemas" (
   "id" SERIAL PRIMARY KEY,
   "fecha_envio" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
   "reportador_id" INTEGER REFERENCES "reportador"("id") ON DELETE SET NULL,
   "activo" BOOLEAN DEFAULT TRUE
);

CREATE TABLE IF NOT EXISTS "detalle_problema" (
   "id" SERIAL PRIMARY KEY,
   "descripcion" TEXT NOT NULL,
   "estado_actual" VARCHAR(50) NOT NULL DEFAULT 'Pendiente',
   "ambiente_id" INTEGER, -- Atributo de referencia (Ubicación)
   "nota_id" INTEGER REFERENCES "nota_problemas"("id") ON DELETE CASCADE,
   "tipo_incidencia_id" INTEGER REFERENCES "tipo_incidencia"("id") ON DELETE SET NULL,
   "activo" BOOLEAN DEFAULT TRUE
);
```

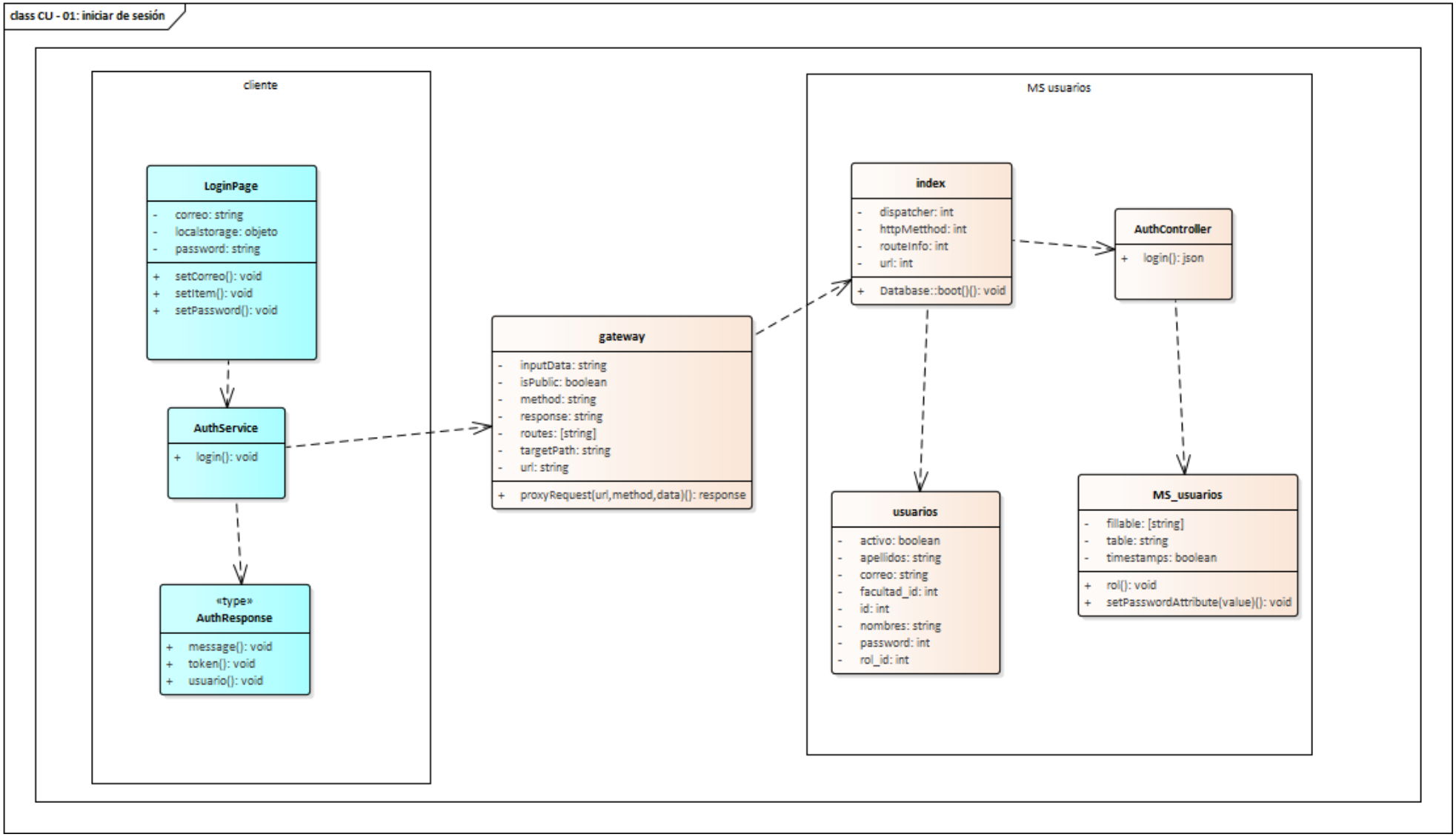
Mantenimiento

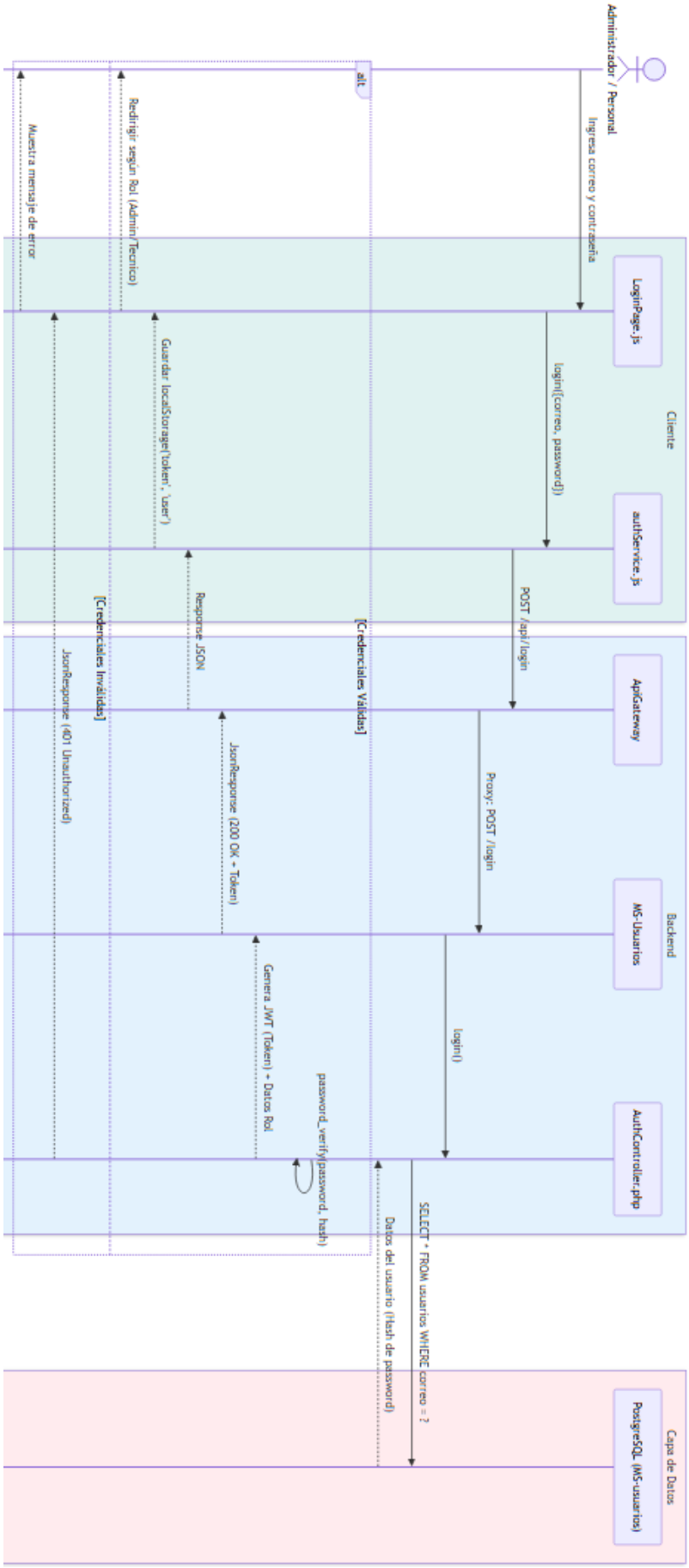
```
CREATE TABLE IF NOT EXISTS "asignacion" (
   "id" SERIAL PRIMARY KEY,
   "fecha_asignacion" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
   "detalle_problema_id" INTEGER, -- Atributo de referencia (Reporte de Problemas)
```

```
    "usuario_id" INTEGER,         -- Atributo de referencia (Usuarios - Técnico)
    "activo" BOOLEAN DEFAULT TRUE
);

CREATE TABLE IF NOT EXISTS "historial_estados" (
    "id" SERIAL PRIMARY KEY,
    "tipo" VARCHAR(50) NOT NULL, -- Ej: Cambio de Estado, Comentario
    "estado" VARCHAR(50) NOT NULL, -- Ej: Asignado, En Proceso, Resuelto
    "fecha_cambio" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "comentario_tecnico" TEXT,
    "asignacion_id" INTEGER REFERENCES "asignacion"("id") ON DELETE CASCADE,
    "activo" BOOLEAN DEFAULT TRUE
);

CREATE TABLE IF NOT EXISTS "evidencia" (
    "id" SERIAL PRIMARY KEY,
    "url_archivo" VARCHAR(255) NOT NULL,
    "momento" VARCHAR(20) NOT NULL, -- Antes, Despues
    "asignacion_id" INTEGER REFERENCES "asignacion"("id") ON DELETE CASCADE,
    "activo" BOOLEAN DEFAULT TRUE
);
```

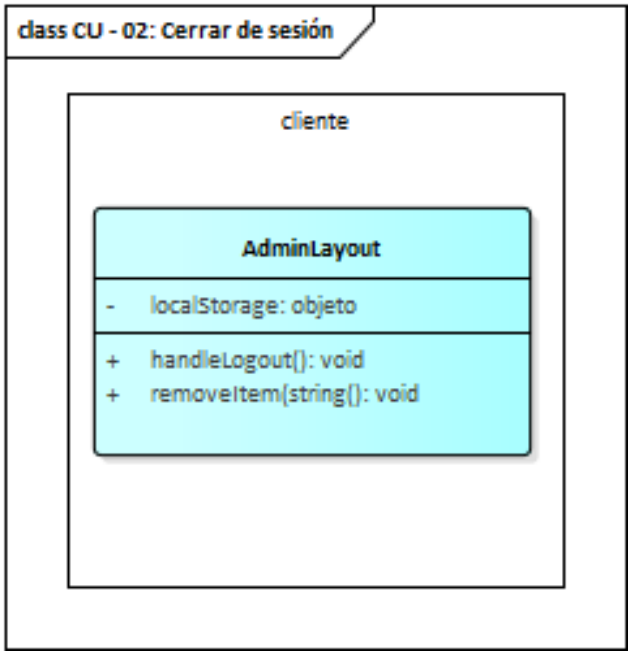
3.3. Diseño de detalle procidemental

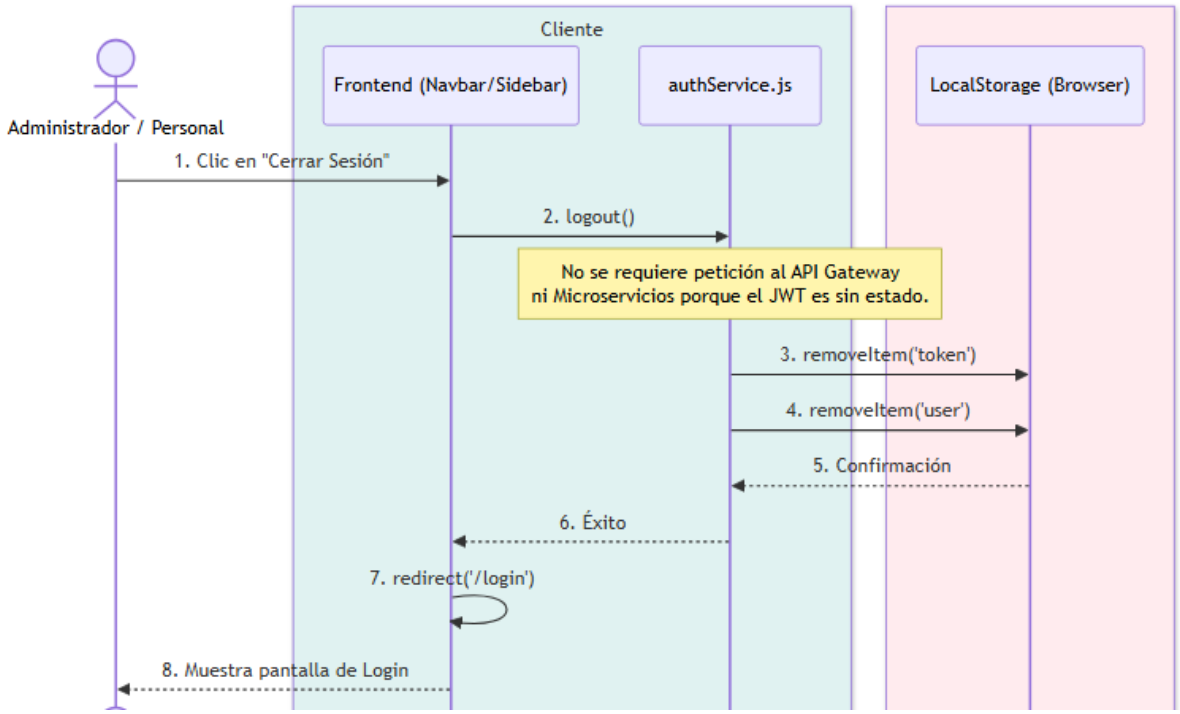
CU – 01: Inicio de sesión

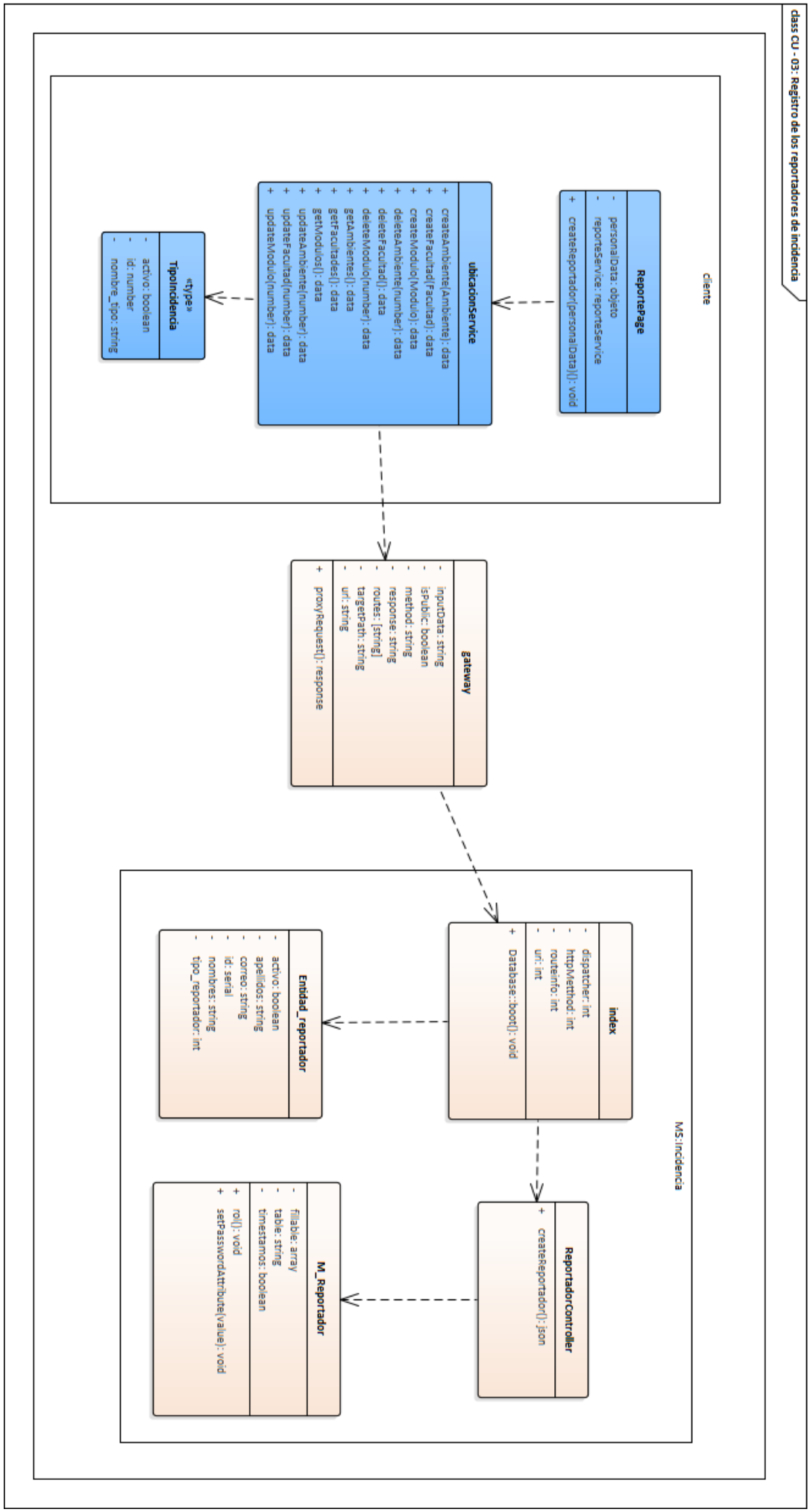

class CU - 01: iniciar de sesión
cliente
LoginPage
- correo: string
- localstorage: objeto
- password: string
+ setCorreo(): void
+ setItem(): void
+ setPassword(): void
AuthService
+ login(): void
«type»
AuthResponse
+ message(): void
+ token(): void
+ usuario(): void
gateway
- inputData: string
- isPublic: boolean
- method: string
- response: string
- routes: [string]
- targetPath: string
- url: string
+ proxyRequest(url,method,data)(): response
MS usuarios
index
- dispatcher: int
- httpMetthod: int
- routeInfo: int
- url: int
+ Database::boot()(): void
AuthController
+ login(): json
usuarios
- activo: boolean
- apellidos: string
- correo: string
- facultad_id: int
- id: int
- nombres: string
- password: int
- rol_id: int
MS_usuarios
- fillable: [string]
- table: string
- timestamps: boolean
+ rol(): void
+ setPasswordAttribute(value)(): void


Administrador / Personal
Cliente
LoginPage.js
authService.js
Backend
ApiGateway
MS-Usuarios
AuthController.php
Capa de Datos
PostgreSQL (MS-usuarios)
Ingresa correo y contraseña
login({correo, password})
POST /api/login
Proxy: POST /login
login()
SELECT * FROM usuarios WHERE correo = ?
Datos del usuario (Hash de password)
password_verify(password, hash)
Genera JWT (Token) + Datos Rol
JsonResponse (200 OK + Token)
Response JSON
Guardar localStorage('token', 'user')
Redirigir según Rol (Admin/Tecnico)
alt
[Credenciales Válidas]
[Credenciales Inválidas]
JsonResponse (401 Unauthorized)
Muestra mensaje de error

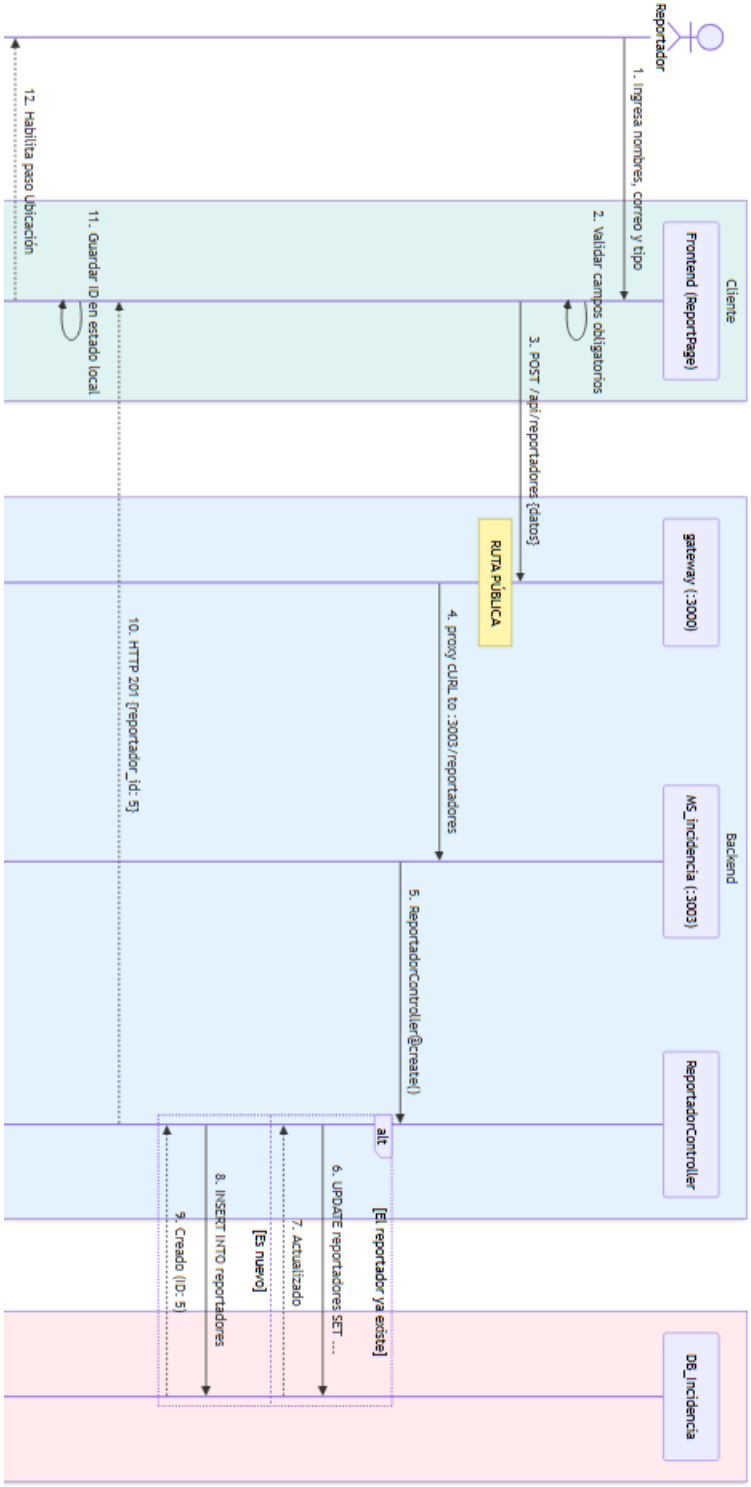
CU-02: Cierre de sesión

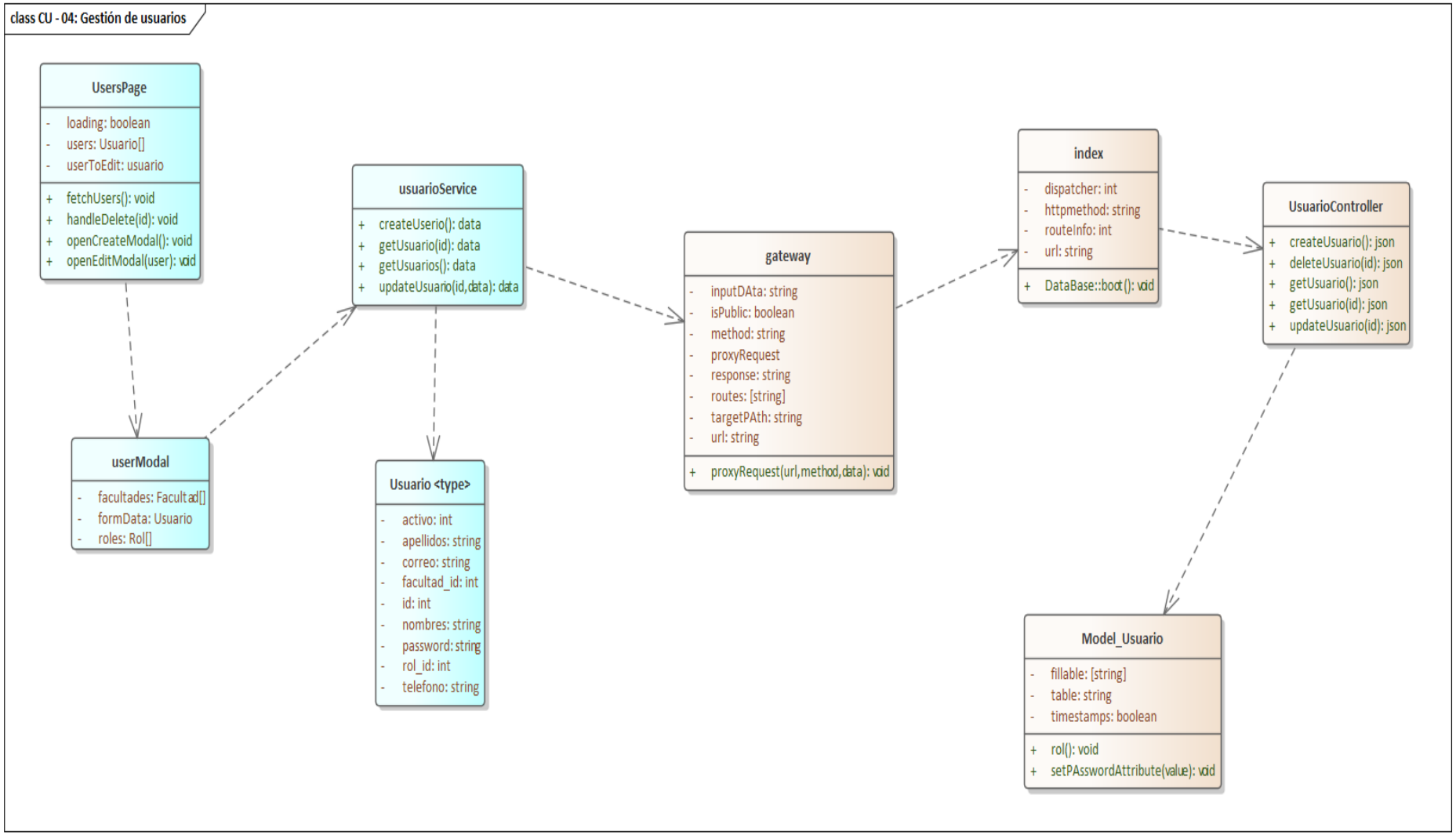




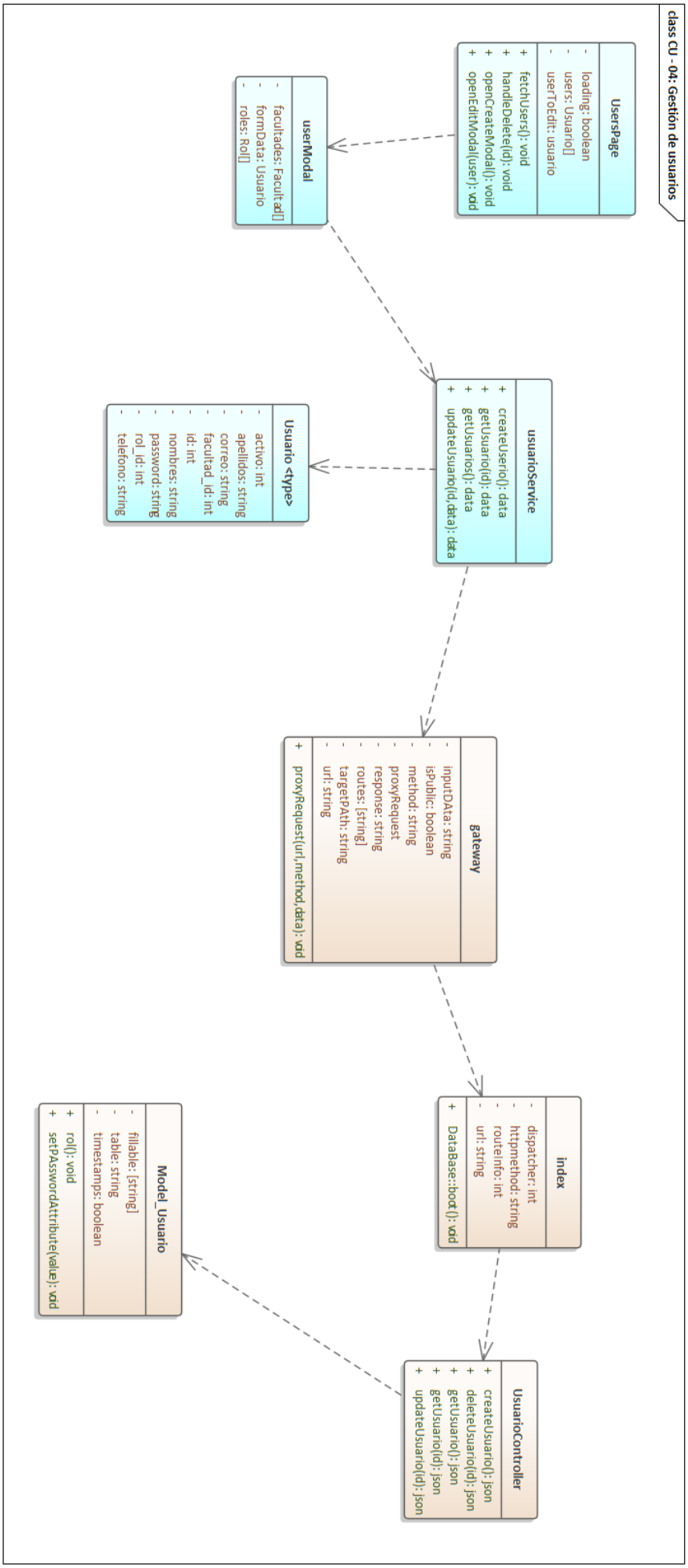


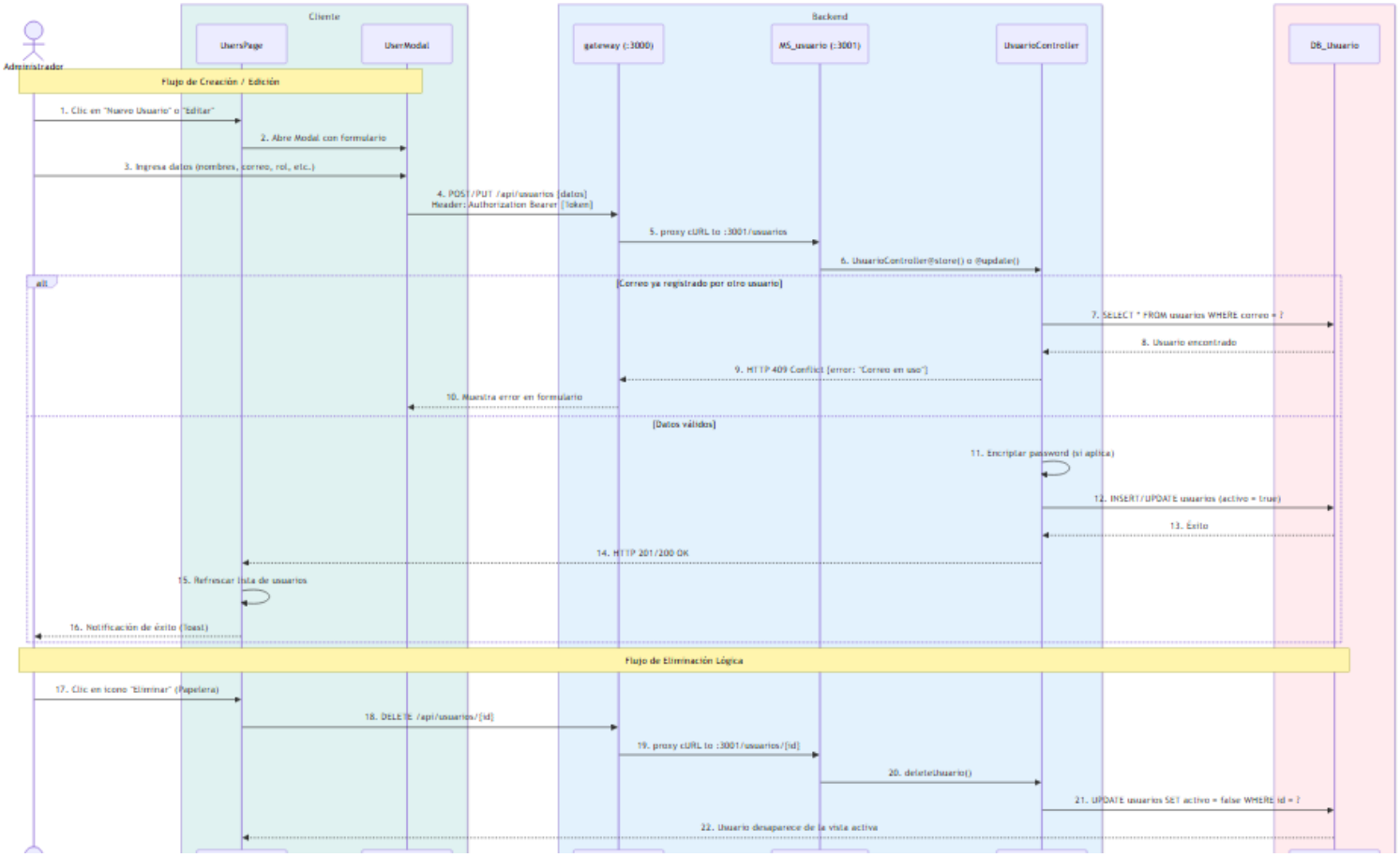
CU-03: Registro de los reportadores de problemas

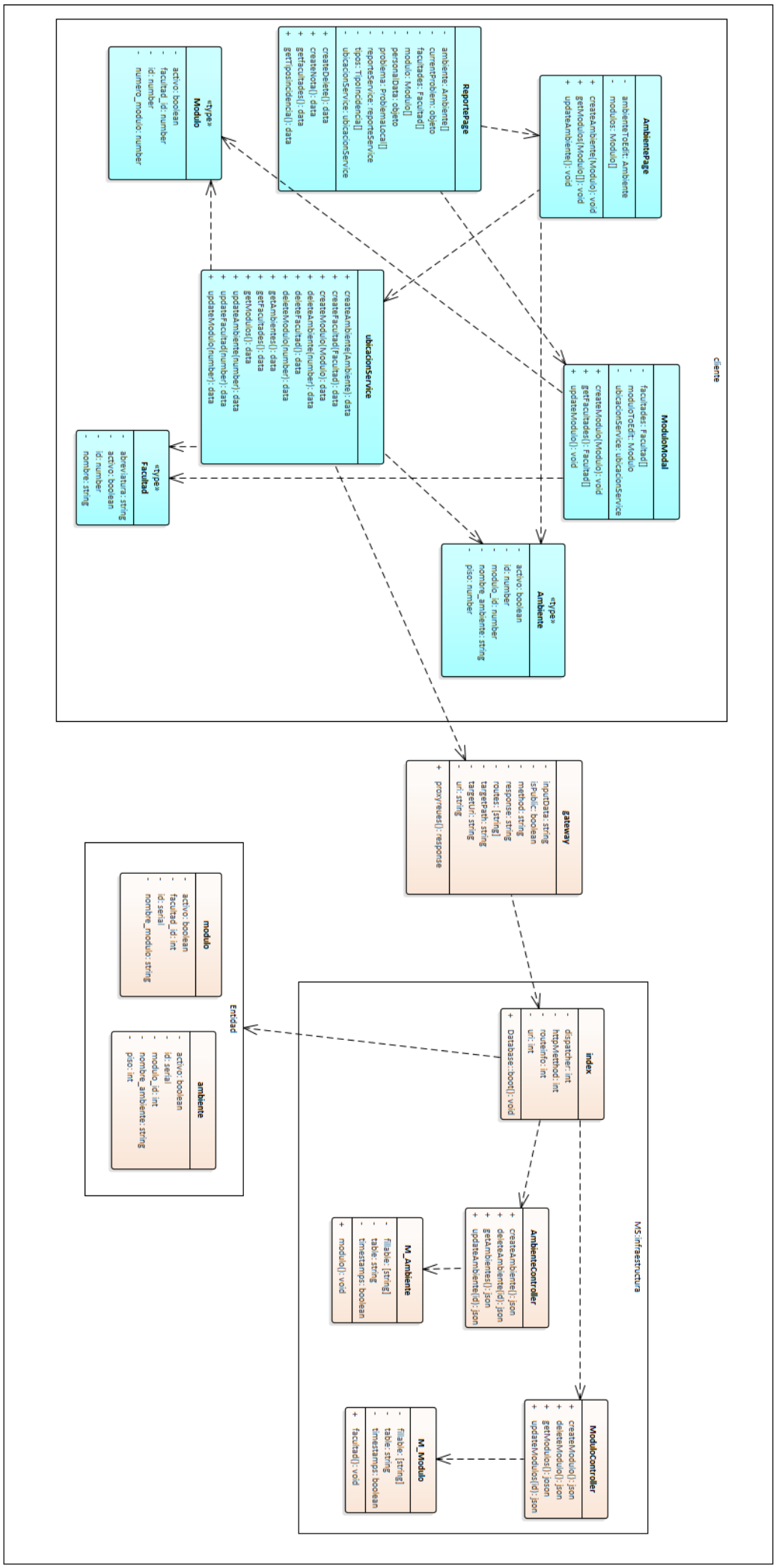

Reportador
Cliente
Frontend (ReportPage)
1. Ingresa nombres, correo y tipo
2. Validar campos obligatorios
3. POST /api/reportadores {datos}
Backend
gateway (:3000)
RUTA PUBLICA
4. proxy cURL to :3003/reportadores
MS_incidencia (:3003)
5. ReportadorController@create()
ReportadorController
alt
[El reportador ya existe]
6. UPDATE reportadores SET ...
7. Actualizado
[Es nuevo]
8. INSERT INTO reportadores
9. Creado (ID: 5)
DB_Incidencia
10. HTTP 201 {reportador_id: 5}
11. Guardar ID en estado local
12. Habilita paso Ubicación


class CU - 04: Gestión de usuarios
UsersPage
- loading: boolean
- users: Usuario[]
- userToEdit: usuario
+ fetchUsers(): void
+ handleDelete(id): void
+ openCreateModal(): void
+ openEditModal(user): void
usuarioService
+ createUserio(): data
+ getUsuario(id): data
+ getUsuarios(): data
+ updateUsuario(id,data): data
gateway
- inputDAta: string
- isPublic: boolean
- method: string
- proxyRequest
- response: string
- routes: [string]
- targetPAth: string
- url: string
+ proxyRequest(url,method,data): void
index
- dispatcher: int
- httpmethod: string
- routeInfo: int
- url: string
+ DataBase::boot(): void
UsuarioController
+ createUsuario(): json
+ deleteUsuario(id): json
+ getUsuario(): json
+ getUsuario(id): json
+ updateUsuario(id): json
userModal
- facultades: Facultad[]
- formData: Usuario
- roles: Rol[]
Usuario <type>
- activo: int
- apellidos: string
- correo: string
- facultad_id: int
- id: int
- nombres: string
- password: string
- rol_id: int
- telefono: string
Model_Usuario
- fillable: [string]
- table: string
- timestamps: boolean
+ rol(): void
+ setPAsswordAttribute(value): void

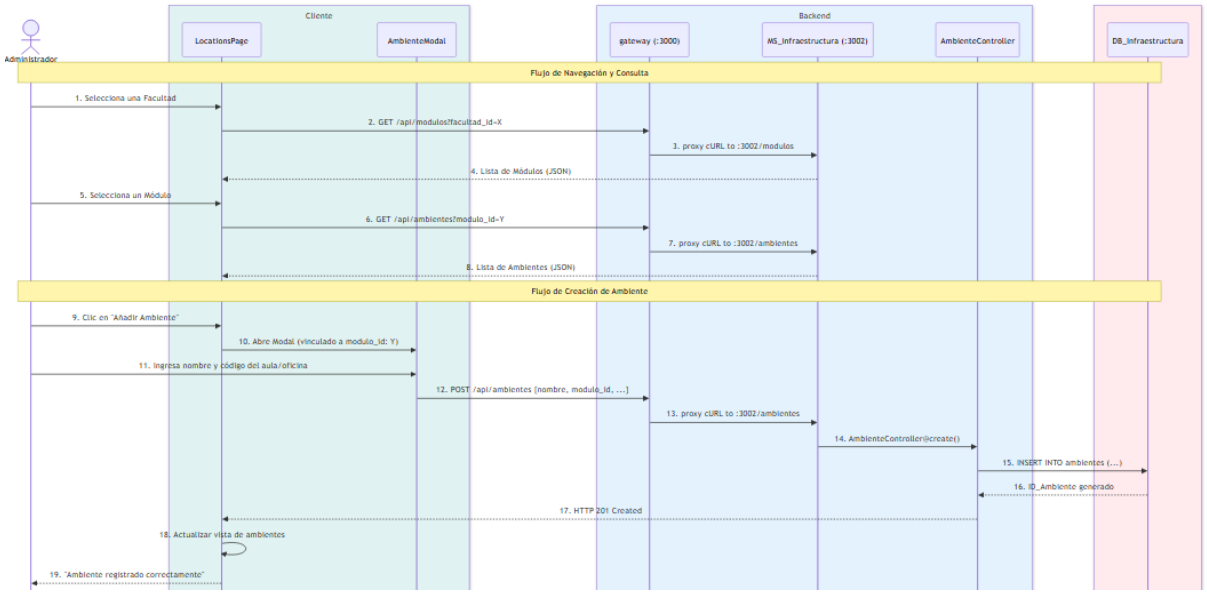
CU-04:Gestion de usuarios

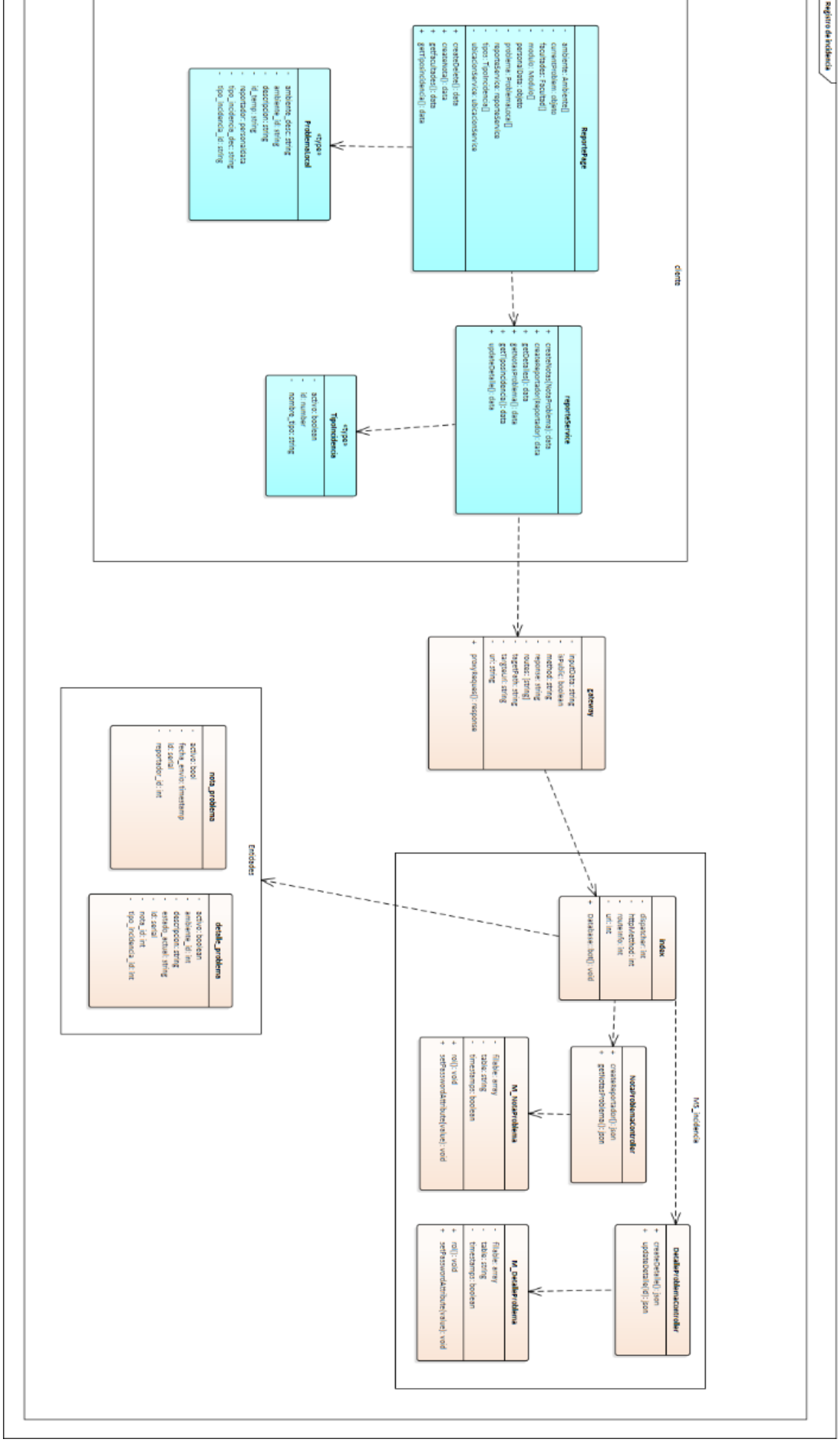



Cliente
Backend
Administrador
UsersPage
UserModal
gateway (:3000)
MS_usuario (:3001)
UsuarioController
DB_Usuario
Flujo de Creación / Edición
1. Clic en "Nuevo Usuario" o "Editar"
2. Abre Modal con formulario
3. Ingresa datos (nombres, correo, rol, etc.)
4. POST/PUT /api/usuarios {datos}
Header: Authorization Bearer {token}
5. proxy cURL to :3001/usuarios
6. UsuarioController@store() o @update()
alt
[Correo ya registrado por otro usuario]
7. SELECT * FROM usuarios WHERE correo = ?
8. Usuario encontrado
9. HTTP 409 Conflict {error: "Correo en uso"}
10. Muestra error en formulario
[Datos válidos]
11. Encriptar password (si aplica)
12. INSERT/UPDATE usuarios (activo = true)
13. Éxito
14. HTTP 201/200 OK
15. Refrescar lista de usuarios
16. Notificación de éxito (toast)
Flujo de Eliminación Lógica
17. Clic en icono "Eliminar" (Papelera)
18. DELETE /api/usuarios/{id}
19. proxy cURL to :3001/usuarios/{id}
20. deleteUsuario()
21. UPDATE usuarios SET activo = false WHERE id = ?
22. Usuario desaparece de la vista activa

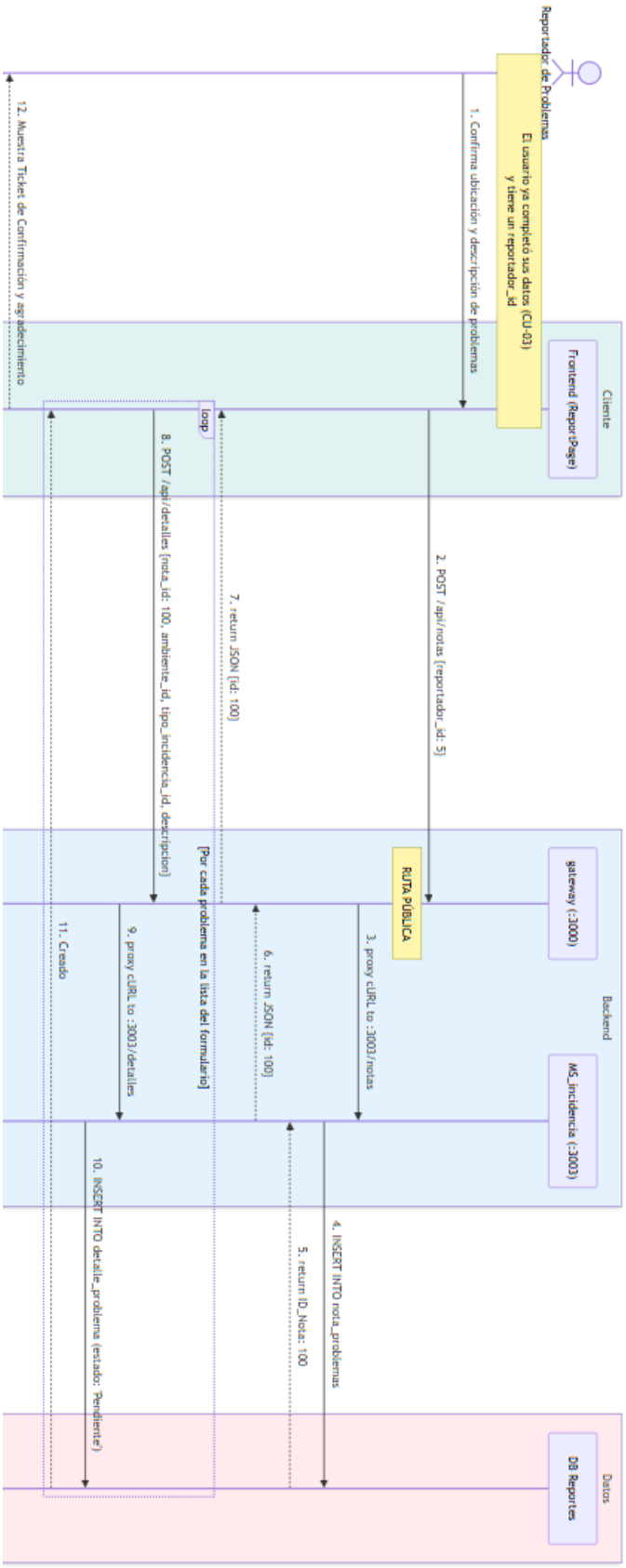


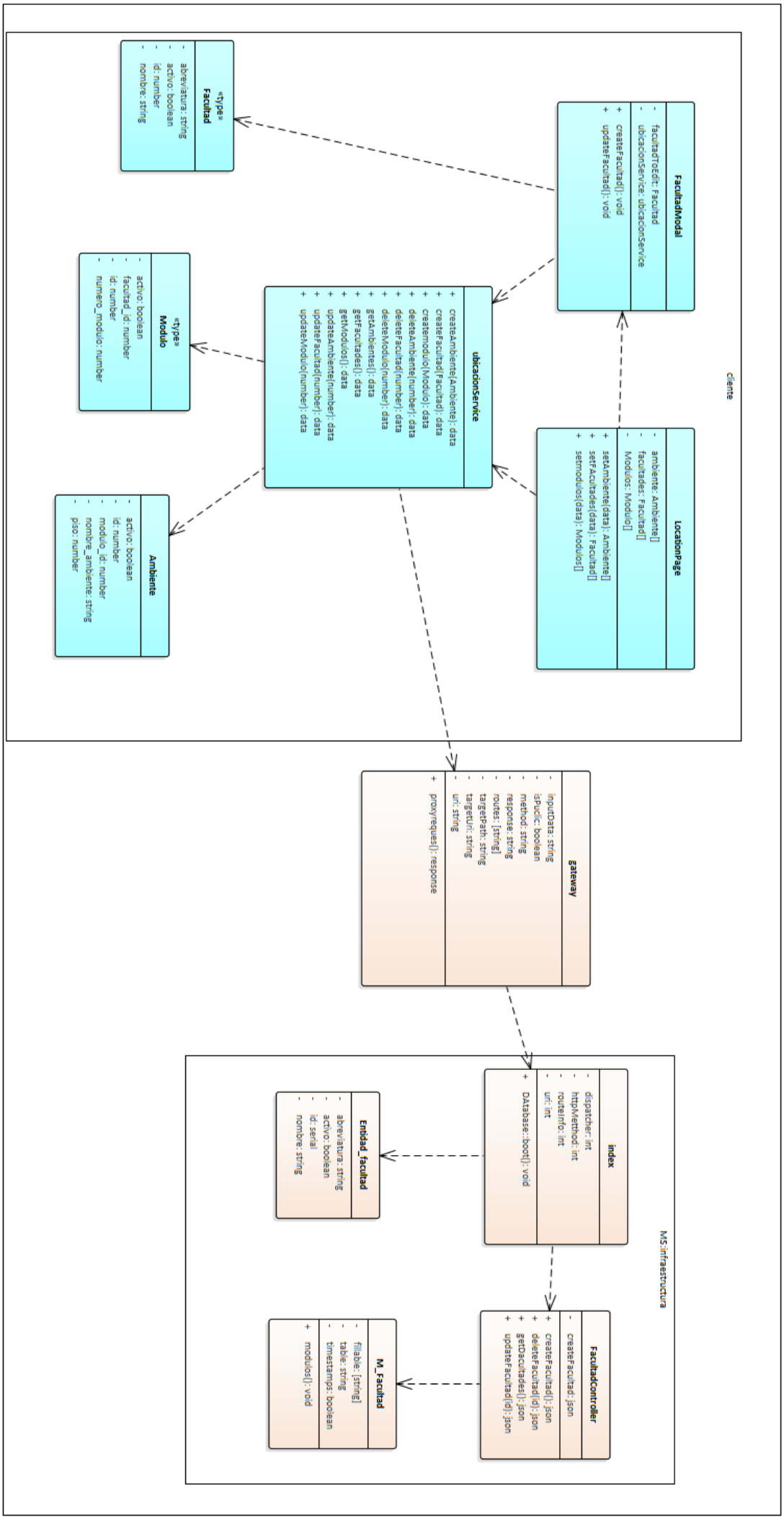
CU-05:gestionar estructura física


Cliente
Backend
Administrador
LocationsPage
AmbienteModal
gateway (:3000)
MS_Infraestructura (:3002)
AmbienteController
DB_Infraestructura
Flujo de Navegación y Consulta
1. Selecciona una Facultad
2. GET /api/modulos?facultad_id=X
3. proxy cURL to :3002/modulos
4. Lista de Módulos (JSON)
5. Selecciona un Módulo
6. GET /api/ambientes?modulo_id=Y
7. proxy cURL to :3002/ambientes
8. Lista de Ambientes (JSON)
Flujo de Creación de Ambiente
9. Clic en "Añadir Ambiente"
10. Abre Modal (vinculado a modulo_id: Y)
11. Ingresa nombre y código del aula/oficina
12. POST /api/ambientes {nombre, module_id, ...}
13. proxy cURL to :3002/ambientes
14. AmbienteController@create()
15. INSERT INTO ambientes (...)
16. ID_Ambiente generado
17. HTTP 201 Created
18. Actualizar vista de ambientes
19. "Ambiente registrado correctamente"

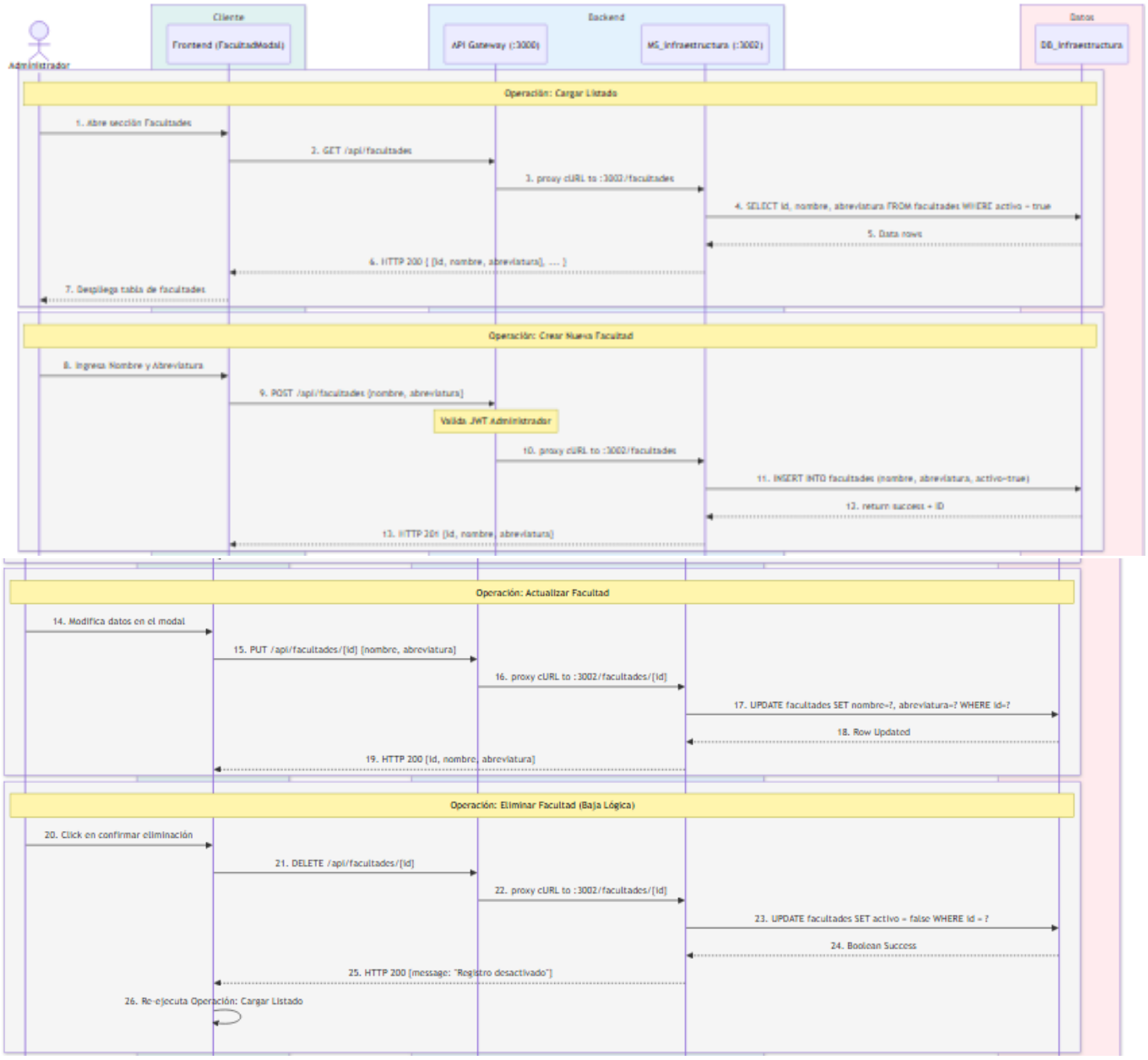


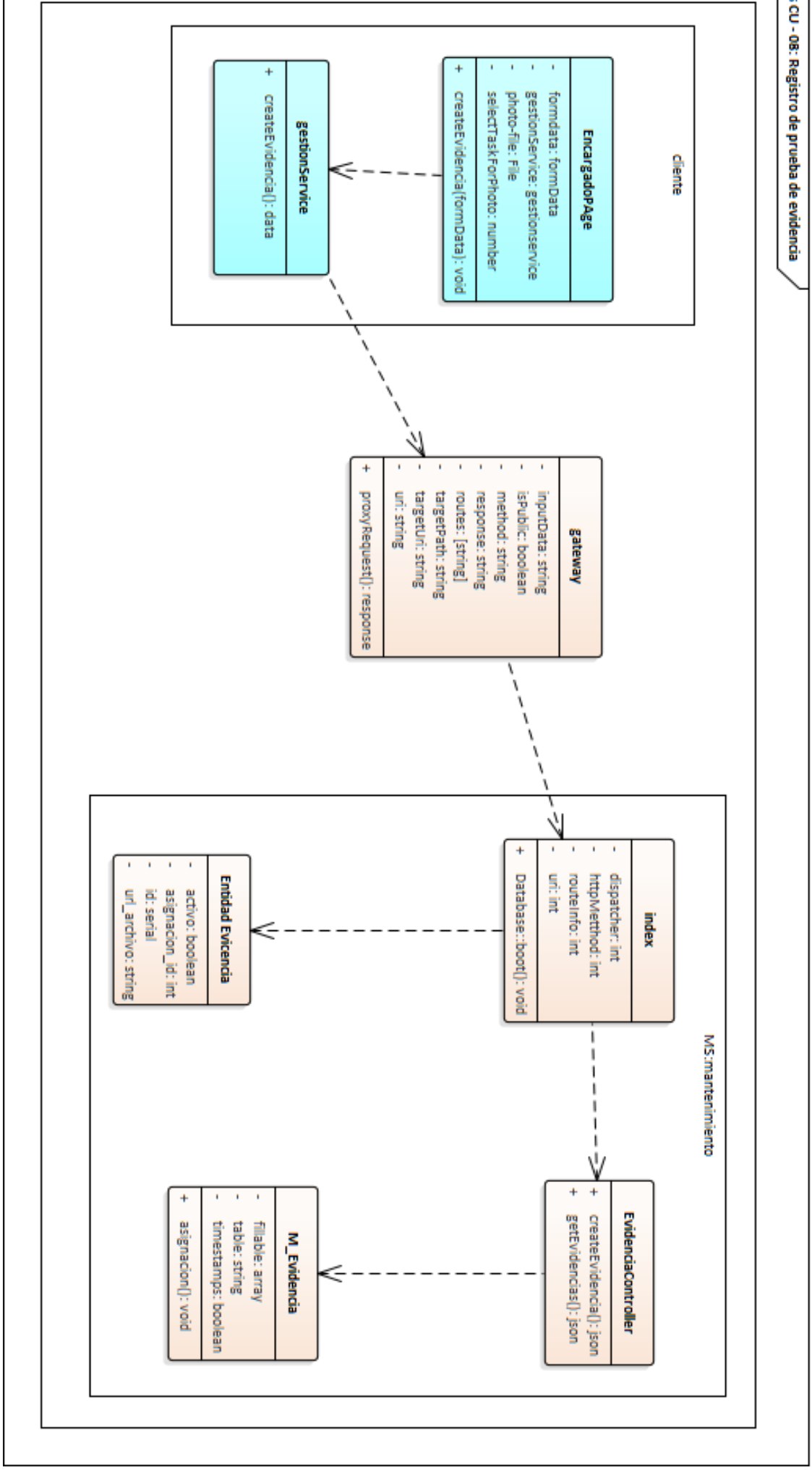
CU-06:registro de notas de problemas


Reportador de Problemas
El usuario ya completó sus datos (CU-03)
y tiene un reportador_id
Cliente
Frontend (ReporterPage)
1. Confirma ubicación y descripción de problemas
2. POST /api/notas {reportador_id: 5}
Backend
gateway (:3000)
RUTA PÚBLICA
3. proxy cURL to :3003/notas
MS_incidencia (:3003)
4. INSERT INTO nota_problemas
5. return ID_Nota: 100
Datos
DB Reportes
6. return JSON {id: 100}
7. return JSON {id: 100}
loop
[Por cada problema en la lista del formulario]
8. POST /api/detalles {nota_id: 100, ambiente_id, tipo_incidencia_id, descripcion}
9. proxy cURL to :3003/detalles
10. INSERT INTO detalle_problema (estado: 'Pendiente')
11. Creado
12. Muestra Ticket de Confirmación y agradecimiento

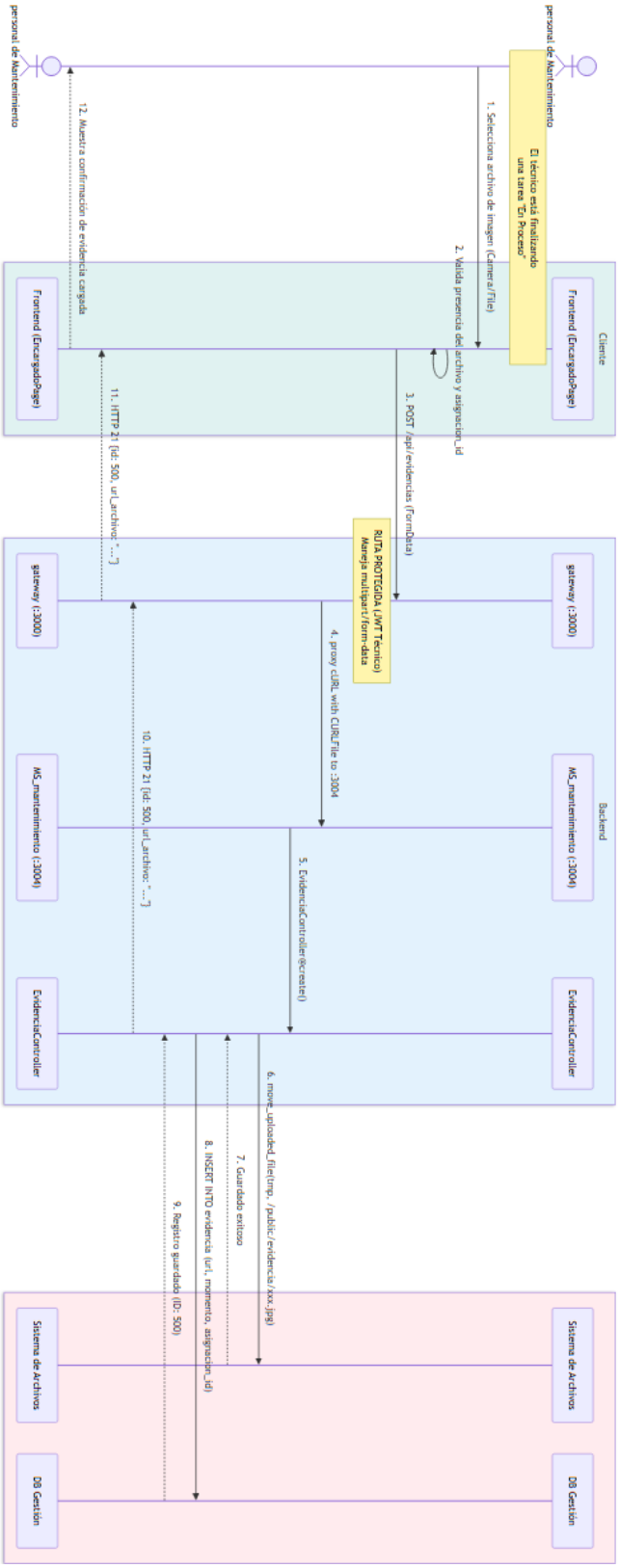


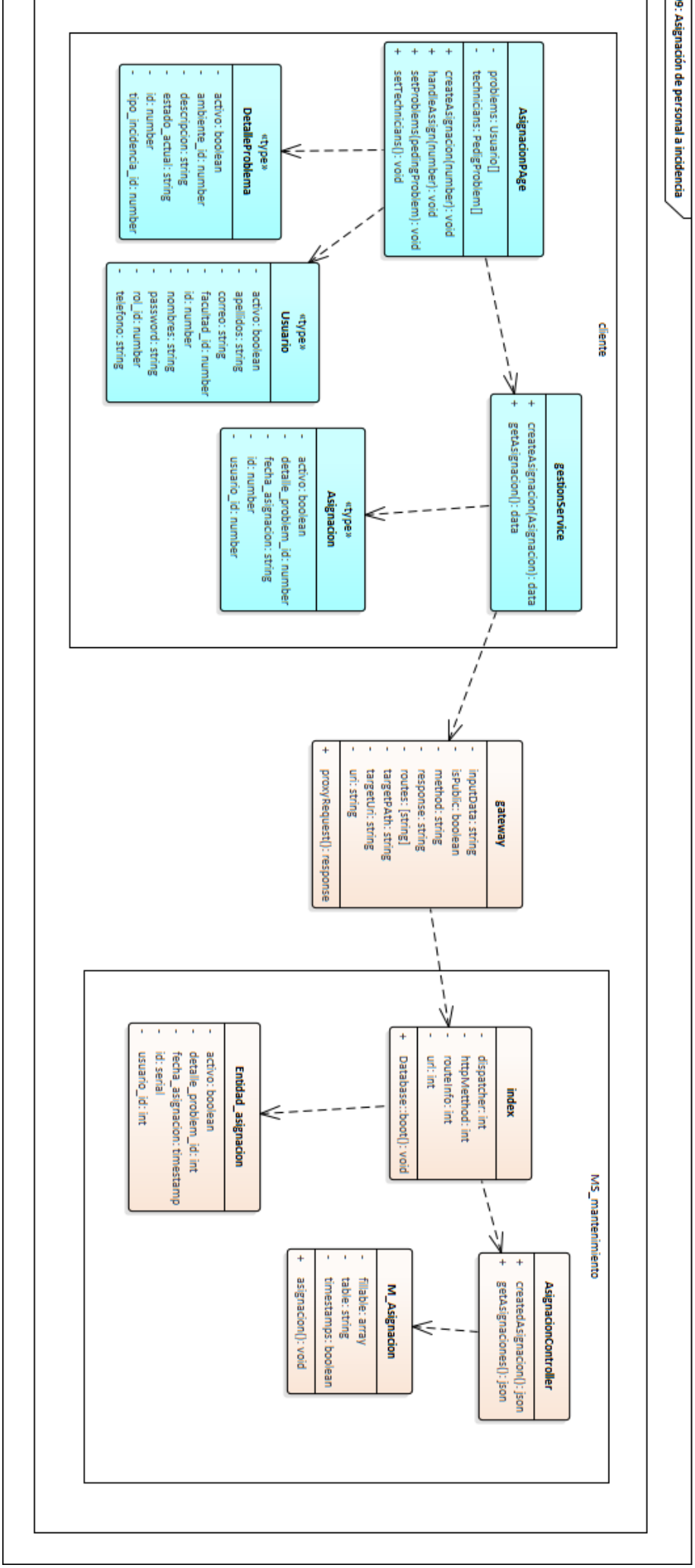
CU-07:gestionar facultades


Administrador
Cliente
Frontend (FacultadModal)
Backend
API Gateway (:3000)
MS_infraestructura (:3002)
Datos
DB_infraestructura
Operación: Cargar Listado
1. Abre sección Facultades
2. GET /api/facultades
3. proxy cURL to :3002/facultades
4. SELECT id, nombre, abreviatura FROM facultades WHERE activo = true
5. Data rows
6. HTTP 200 [{id, nombre, abreviatura}, ...]
7. Despliega tabla de facultades
Operación: Crear Nueva Facultad
8. Ingresa Nombre y Abreviatura
9. POST /api/facultades {nombre, abreviatura}
Valida JWT Administrador
10. proxy cURL to :3002/facultades
11. INSERT INTO facultades (nombre, abreviatura, activo=true)
12. return success + ID
13. HTTP 201 {id, nombre, abreviatura}
Operación: Actualizar Facultad
14. Modifica datos en el modal
15. PUT /api/facultades/{id} {nombre, abreviatura}
16. proxy cURL to :3002/facultades/{id}
17. UPDATE facultades SET nombre=?, abreviatura=? WHERE id=?
18. Row Updated
19. HTTP 200 {id, nombre, abreviatura}
Operación: Eliminar Facultad (Baja Lógica)
20. Click en confirmar eliminación
21. DELETE /api/facultades/{id}
22. proxy cURL to :3002/facultades/{id}
23. UPDATE facultades SET activo = false WHERE id = ?
24. Boolean Success
25. HTTP 200 {message: "Registro desactivado"}
26. Re-ejecuta Operación: Cargar Listado

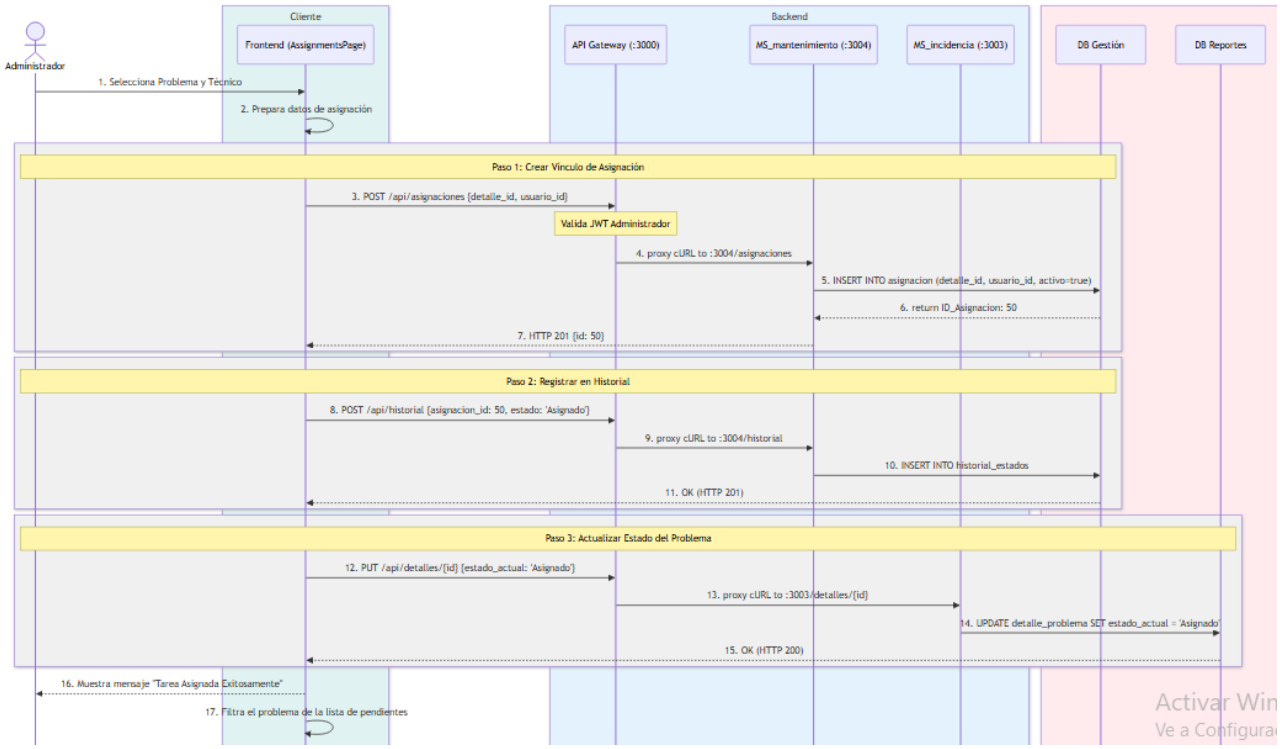


CU08:registro de evidencia

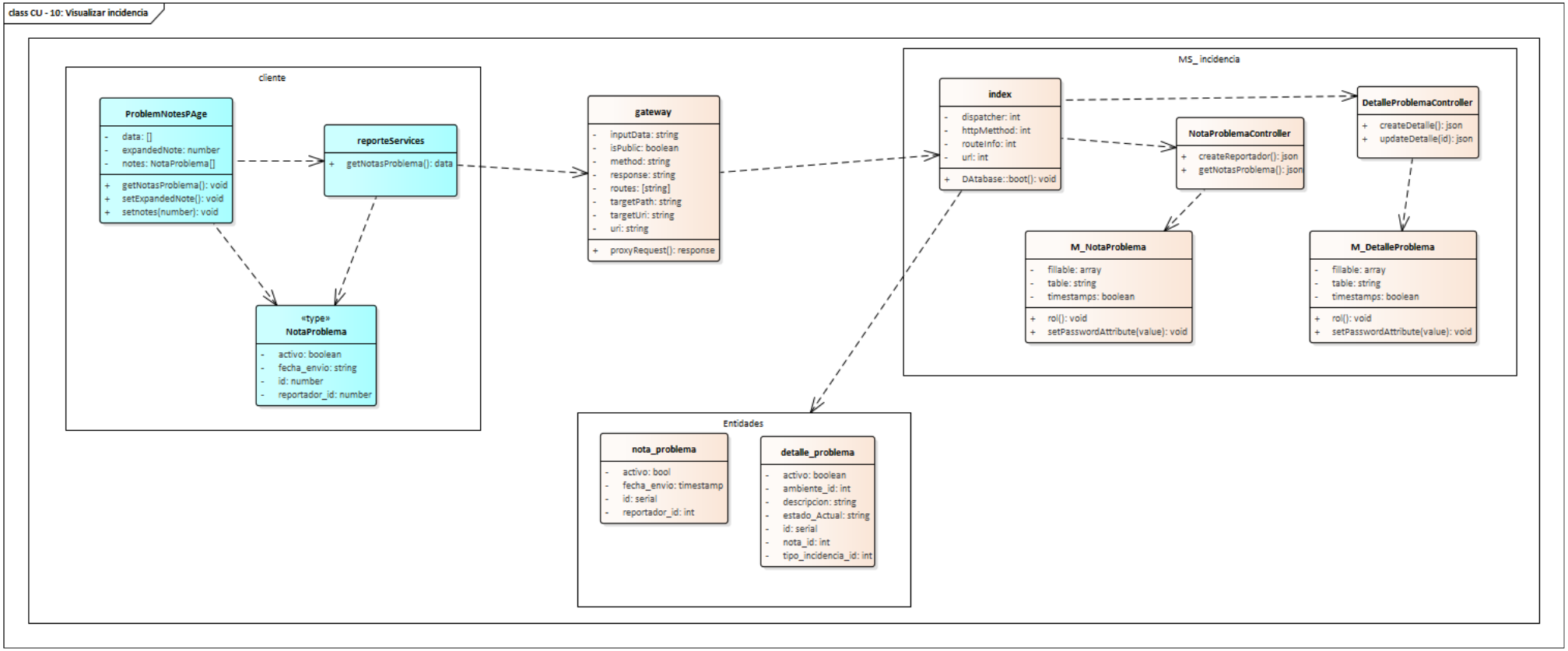

personal de Mantenimiento
personal de Mantenimiento
El técnico está finalizando
una tarea "En Proceso"
Cliente
Frontend (EncargadoPage)
Frontend (EncargadoPage)
1. Selecciona archivo de imagen (Camera/File)
2. Valida presencia del archivo y asignacion_id
3. POST /api/evidencias (FormData)
RUTA PROTEGIDA (JWT Tecnico)
Maneja multipart/form-data
Backend
gateway (:3000)
gateway (:3000)
4. proxy cURL with CURLFile to :3004
MS_mantenimiento (:3004)
MS_mantenimiento (:3004)
5. EvidenciaController@create()
EvidenciaController
EvidenciaController
6. move_uploaded_file(tmp, /public/evidencia/xxx.jpg)
7. Guardado exitoso
8. INSERT INTO evidencia (url, momento, asignacion_id)
9. Registro guardado (ID: 500)
10. HTTP 21 {id: 500, url_archivo: "..."}
11. HTTP 21 {id: 500, url_archivo: "..."}
12. Muestra confirmación de evidencia cargada
Sistema de Archivos
Sistema de Archivos
DB Gestión
DB Gestión

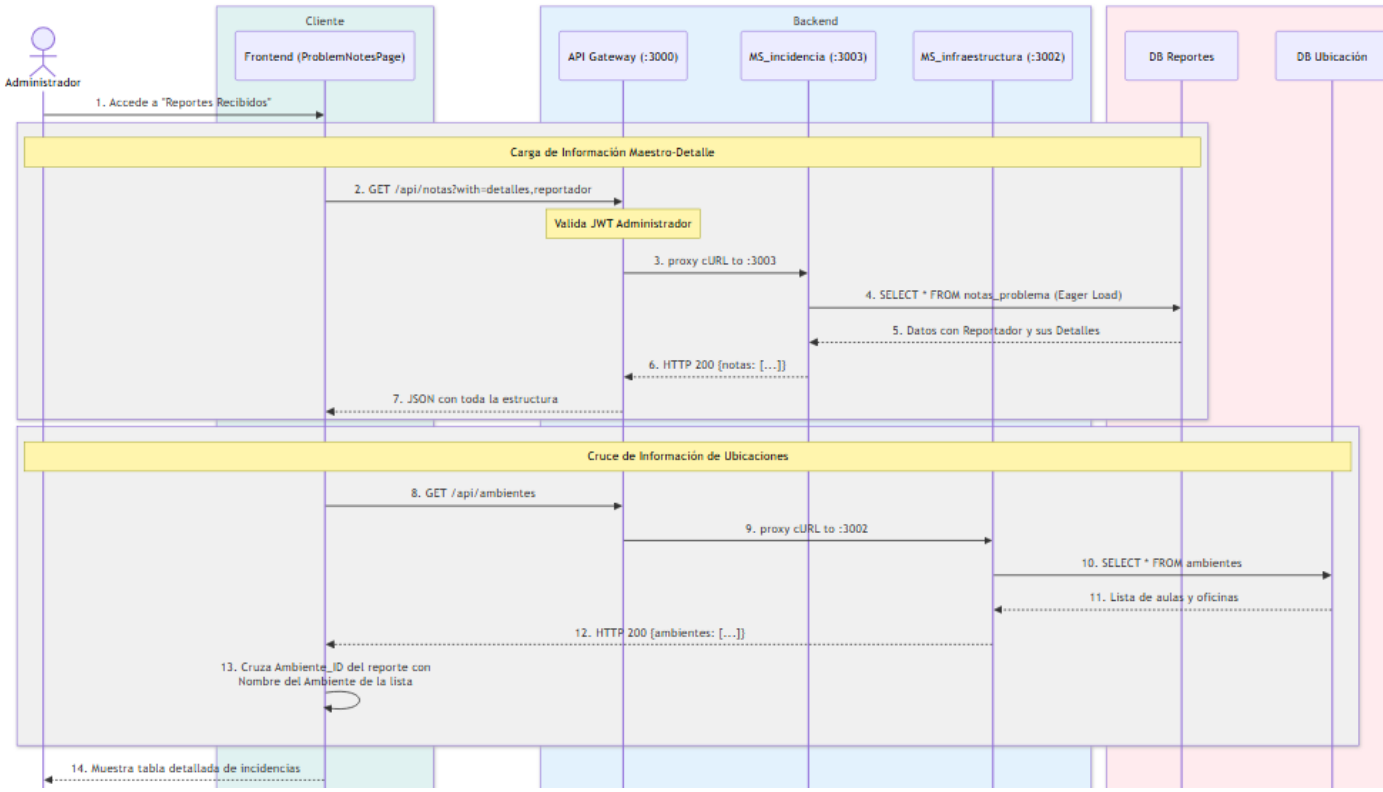


CU09:asignación de problema

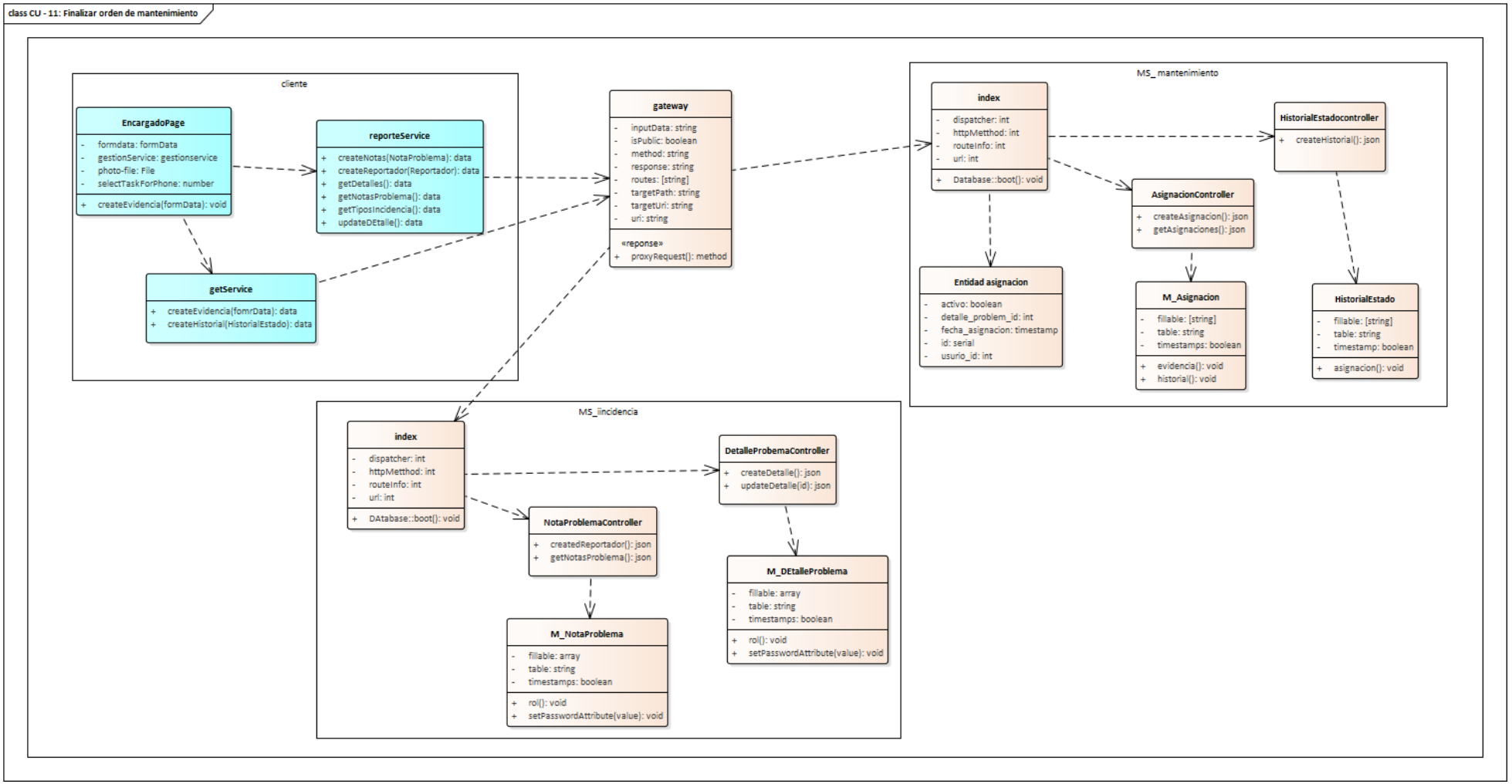


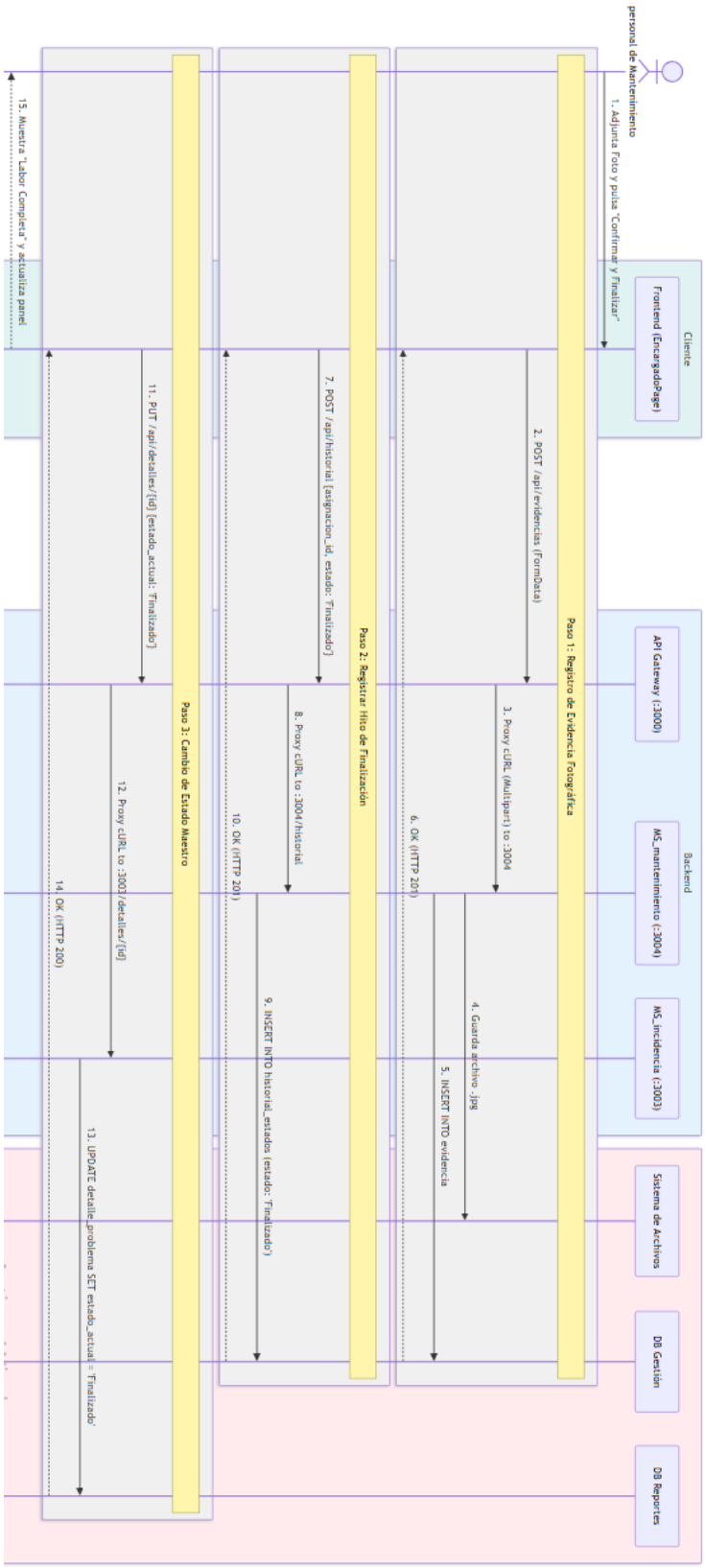
CU10:visualizar problemas

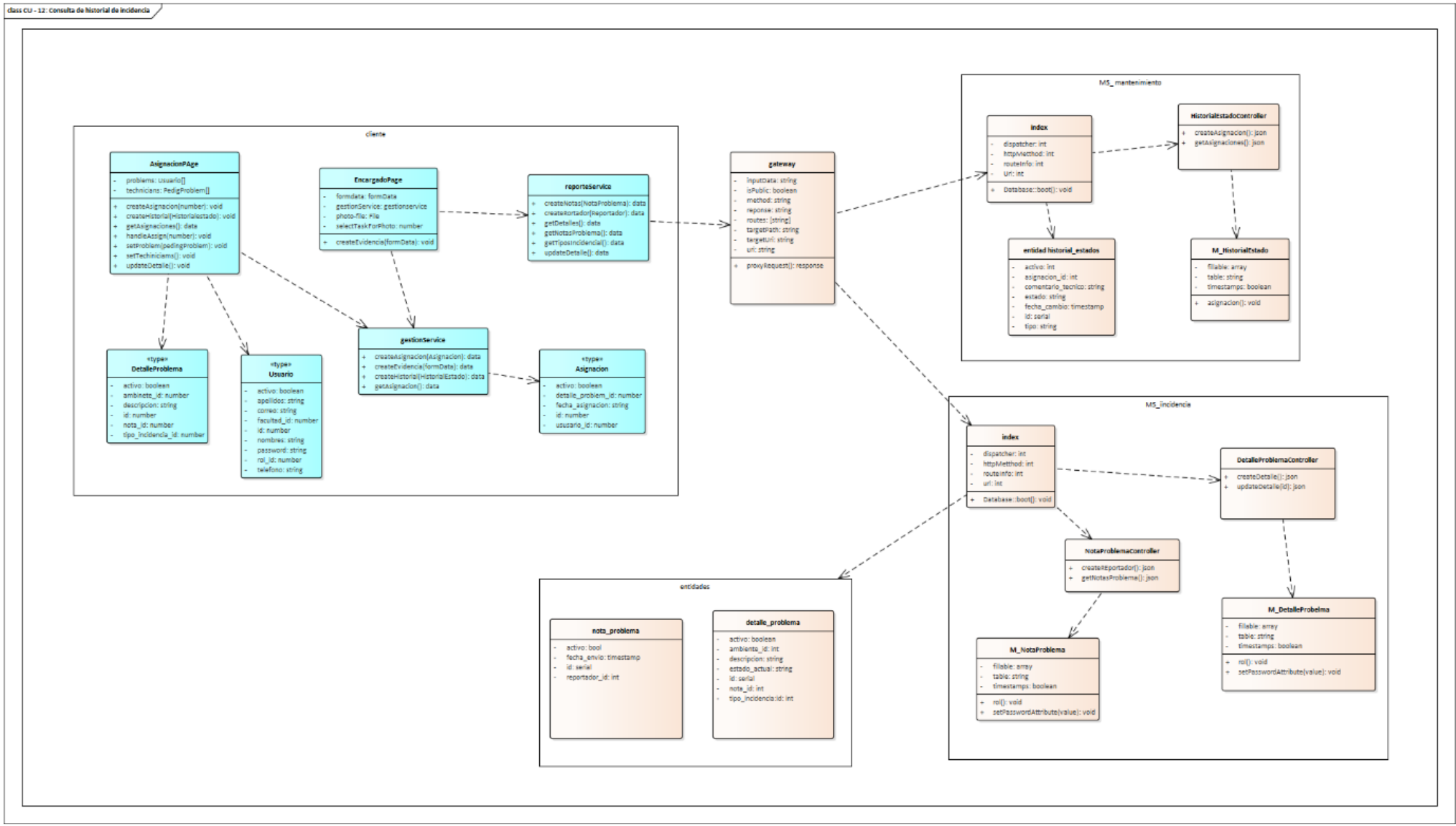

class CU - 10: Visualizar incidencia
cliente
ProblemNotesPAge
- data: []
- expandedNote: number
- notes: NotaProblema[]
+ getNotasProblema(): void
+ setExpandedNote(): void
+ setnotes(number): void
reporteServices
+ getNotasProblema(): data
«type»
NotaProblema
- activo: boolean
- fecha_envio: string
- id: number
- reportador_id: number
gateway
- inputData: string
- isPublic: boolean
- method: string
- response: string
- routes: [string]
- targetPath: string
- targetUri: string
- uri: string
+ proxyRequest(): response
MS_ incidencia
index
- dispatcher: int
- httpMetthod: int
- routeInfo: int
- url: int
+ DAtabase::boot(): void
NotaProblemaController
+ createReportador(): json
+ getNotasProblema(): json
DetalleProblemaController
+ createDetalle(): json
+ updateDetalle(id): json
M_NotaProblema
- fillable: array
- table: string
- timestamps: boolean
+ rol(): void
+ setPasswordAttribute(value): void
M_DetalleProblema
- fillable: array
- table: string
- timestamps: boolean
+ rol(): void
+ setPasswordAttribute(value): void
Entidades
nota_problema
- activo: bool
- fecha_envio: timestamp
- id: serial
- reportador_id: int
detalle_problema
- activo: boolean
- ambiente_id: int
- descripcion: string
- estado_Actual: string
- id: serial
- nota_id: int
- tipo_incidencia_id: int



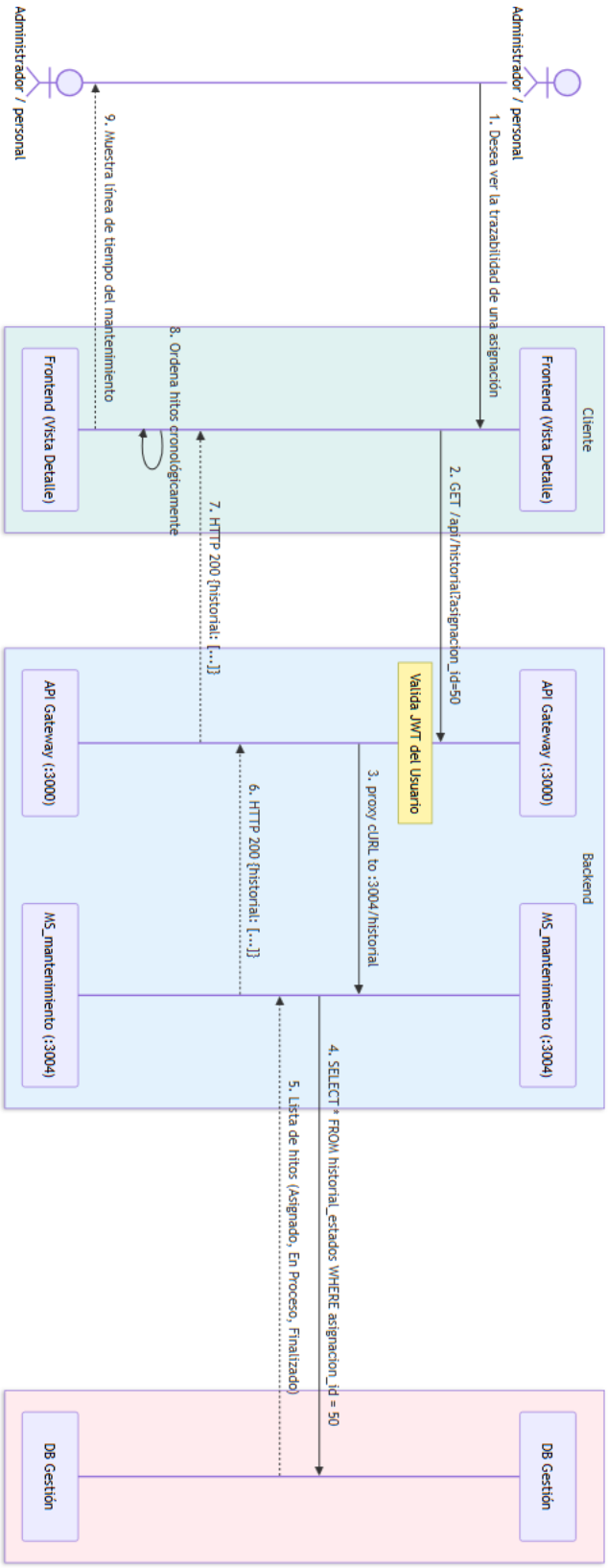
CU11:Finalizar mantenimiento


class CU - 11: Finalizar orden de mantenimiento
cliente
EncargadoPage
- formdata: formData
- gestionService: gestionservice
- photo-file: File
- selectTaskForPhone: number
+ createEvidencia(formData): void
reporteService
+ createNotas(NotaProblema): data
+ createReportador(Reportador): data
+ getDetalles(): data
+ getNotasProblema(): data
+ getTiposIncidencia(): data
+ updateDEtalle(): data
getService
+ createEvidencia(fomrData): data
+ createHistorial(HistorialEstado): data
gateway
- inputData: string
- isPublic: boolean
- method: string
- response: string
- routes: [string]
- targetPath: string
- targetUri: string
- uri: string
«reponse»
+ proxyRequest(): method
MS_ mantenimiento
index
- dispatcher: int
- httpMetthod: int
- routeInfo: int
- url: int
+ Database::boot(): void
HistorialEstadocontroller
+ createHistorial(): json
AsignacionController
+ createAsignacion(): json
+ getAsignaciones(): json
Entidad asignacion
- activo: boolean
- detalle_problem_id: int
- fecha_asignacion: timestamp
- id: serial
- usurio_id: int
M_Asignacion
- fillable: [string]
- table: string
- timestamps: boolean
+ evidencia(): void
+ historial(): void
HistorialEstado
- fillable: [string]
- table: string
- timestamp: boolean
+ asignacion(): void
MS_iincidencia
index
- dispatcher: int
- httpMetthod: int
- routeInfo: int
- url: int
+ DAtabase::boot(): void
DetalleProbemaController
+ createDetalle(): json
+ updateDetalle(id): json
NotaProblemaController
+ createdReportador(): json
+ getNotasProblema(): json
M_DEtalleProblema
- fillable: array
- table: string
- timestamps: boolean
+ rol(): void
+ setPasswordAttribute(value): void
M_NotaProblema
- fillable: array
- table: string
- timestamps: boolean
+ rol(): void
+ setPasswordAttribute(value): void


personal de Mantenimiento
Cliente
Frontend (EncargadoPage)
Backend
API Gateway (:3000)
MS_mantenimiento (:3004)
MS_incidencia (:3003)
Sistema de Archivos
DB Gestión
DB Reportes
1. Adjunta Foto y pulsa "Confirmar y Finalizar"
Paso 1: Registro de Evidencia Fotográfica
2. POST /api/evidencias (FormData)
3. Proxy cURL (Multipart) to :3004
4. Guarda archivo .jpg
5. INSERT INTO evidencia
6. OK (HTTP 201)
Paso 2: Registrar Hito de Finalización
7. POST /api/historial {asignacion_id, estado: 'Finalizado'}
8. Proxy cURL to :3004/historial
9. INSERT INTO historial_estados (estado: 'Finalizado')
10. OK (HTTP 201)
Paso 3: Cambio de Estado Maestro
11. PUT /api/detalles/{id} {estado_actual: 'Finalizado'}
12. Proxy cURL to :3003/detalles/{id}
13. UPDATE detalle_problema SET estado_actual = 'Finalizado'
14. OK (HTTP 200)
15. Muestra "Labor Completa" y actualiza panel


class CU - 12: Consulta de historial de incidencia
cliente
AsignacionPAge
- problems: usuario[]
- technicians: PedigProblem[]
+ createAsignacion(number): void
+ createHistorial(Historialestado): void
+ getAsignaciones(): data
+ handleAssign(number): void
+ setProblem(podingProblem): void
+ setTechiniciems(): void
+ updateDetalle(): void
EncargadoPage
- formdata: formData
- gestionService: gestionservice
- photo-file: File
- selectTaskForPhoto: number
+ createEvidencia(formData): void
reporteservice
+ createNotas(NotaProblema): data
+ createReportador(Reportador): data
+ getDetalles(): data
+ getNotasProblema(): data
+ getTiposIncidencia(): data
+ updateDetalle(): data
«type»
DetalleProblema
- activo: boolean
- ambiente_id: number
- descripcion: string
- id: number
- nota_id: number
- tipo_incidencia_id: number
«type»
Usuario
- activo: boolean
- apellidos: string
- correo: string
- facultad_id: number
- id: number
- nombres: string
- password: string
- rol_id: number
- telefono: string
gestionService
+ createAsignacion(Asignacion): data
+ createEvidencia(formData): data
+ createHistorial(HistorialEstado): data
+ getAsignacion(): data
«type»
Asignacion
- activo: boolean
- detalle_problem_id: number
- fecha_asignacion: string
- id: number
- ususario_id: number
gateway
- inputData: string
- isPublic: boolean
- method: string
- response: string
- routes: [string]
- targetPath: string
- targetUrl: string
- url: string
+ proxyRequest(): response
MS_mantenimiento
index
- dispatcher: int
- httpMethod: int
- routeInfo: int
- Url: int
+ Database::boot(): void
HistorialEstadoController
+ createAsignacion(): json
+ getAsignaciones(): json
entidad historial_estados
- activo: int
- asignacion_id: int
- comentario_tecnico: string
- estado: string
- fecha_cambio: timestamp
- id: serial
- tipo: string
M_HistorialEstado
- fillable: array
- table: string
- timestamps: boolean
+ asignacion(): void
MS_incidencia
index
- dispatcher: int
- httpMethod: int
- routeInfo: int
- url: int
+ Database::boot(): void
DetalleProblemaController
+ createDetalle(): json
+ updateDetalle(id): json
NotaProblemaController
+ createReportador(): json
+ getNotasProblema(): json
M_NotaProblema
- fillable: array
- table: string
- timestamps: boolean
+ rol(): void
+ setPasswordAttribute(value): void
M_DetalleProbelma
- fillable: array
- table: string
- timestamps: boolean
+ rol(): void
+ setPasswordAttribute(value): void
entidades
nota_problema
- activo: bool
- fecha_envio: timestamp
- id: serial
- reportador_id: int
detalle_problema
- activo: boolean
- ambiente_id: int
- descripcion: string
- estado_actual: string
- id: serial
- nota_id: int
- tipo_incidenciaid: int

CU12:consulta de historial


Administrador / personal
Cliente
Frontend (Vista Detalle)
1. Desea ver la trazabilidad de una asignación
2. GET /api/historial?asignacion_id=50
Backend
API Gateway (:3000)
Valida JWT del Usuario
3. proxy cURL to :3004/historial
MS_mantenimiento (:3004)
4. SELECT * FROM historial_estados WHERE asignacion_id = 50
DB Gestión
5. Lista de hitos (Asignado, En Proceso, Finalizado)
6. HTTP 200 {historial: [...]}
API Gateway (:3000)
7. HTTP 200 {historial: [...]}
Frontend (Vista Detalle)
8. Ordena hitos cronológicamente
9. Muestra línea de tiempo del mantenimiento
Administrador / personal

Código

https://github.com/Jorgito-cc/examen_Arquitectura-software.git